

DAWOOD UNIVERSITY OF ENGINEERING AND TECHNOLOGY

M.A JINNAH ROAD KARACHI-74800 (PAKISTAN)

FACULTY OF INFORMATION & COMPUTING SCIENCES

DEPARTMENT OF ARTIFICIAL INTELLIGENCE

MIKA: A FULLY OFFLINE AND PRIVACY PRESERVING AI ASSISTANT FOR VOICE INTERACTION AND REAL TIME VISUAL PERCEPTION

BACHELOR OF SCIENCE IN ARTIFICIAL INTELLIGENCE (BATCH 2022)

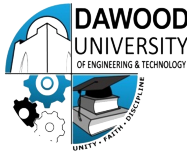
IBAD MOIN (22F-BSAI-58)

MARYAM TARIQ (22F-BSAI-66)

KHUZAIMA IRFAN (22F-BSAI-96)

ABDUL WASAY SARWAR (22F-BSAI-111)

MAY 2026



DAWOOD UNIVERSITY OF ENGINEERING AND TECHNOLOGY

M.A JINNAH ROAD KARACHI-74800 (PAKISTAN)

FACULTY OF INFORMATION & COMPUTING SCIENCES

DEPARTMENT OF ARTIFICIAL INTELLIGENCE

CERTIFICATE

This is to certify that the Final Year Project titled “**Mika: A Fully Offline and Privacy Preserving AI Assistant for Voice Interaction and Real Time Visual Perception**” has been carried out by the following group members under the supervision of **Ms. Noor ul Huda**.

FYP Team Members

Ibad Moin (22F-BSAI-58)

Maryam Tariq (22F-BSAI-66)

Khuzaima Irfan (22F-BSAI-96)

Abdul Wasay Sarwar (22F-BSAI-111)

Supervisor

Ms. Noor ul Huda

Lecturer

FYP Coordinator

Chairperson

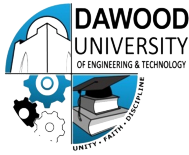
RAHILA SHAH

Lecturer

Dr. ADNAN WAQAR

Associate Professor

Department of Artificial Intelligence



DAWOOD UNIVERSITY OF ENGINEERING AND TECHNOLOGY

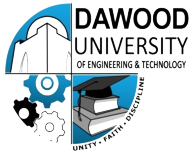
M.A JINNAH ROAD KARACHI-74800 (PAKISTAN)

FACULTY OF INFORMATION & COMPUTING SCIENCES

DEPARTMENT OF ARTIFICIAL INTELLIGENCE

Dedication

This project is, for our parents who we love much. They always prayed for us. Supported us. This has given us the strength to keep going. We also want to thank our teachers for showing us the way to learn things. We want to thank the visually impaired community because we made this project for them. We hope this project will help make their lives a little easier and more independent.



DAWOOD UNIVERSITY OF ENGINEERING AND TECHNOLOGY

M.A JINNAH ROAD KARACHI-74800 (PAKISTAN)

FACULTY OF INFORMATION & COMPUTING SCIENCES

DEPARTMENT OF ARTIFICIAL INTELLIGENCE

ACKNOWLEDGEMENT

First of all we would like to thank **Almighty Allah** for providing us health, wisdom and perseverance to complete this research.

We are thankful to our Supervisor **Ms. Noor ul Huda**, for her guidance and continuous help. Her knowledge in Edge AI and model optimization was very critical in addressing the issues of on-device orchestration and hardware constraints. We are also very thankful to Head of Department **Dr. Adnan Waqar** and FYP Coordinator **Ms. Rahila Shah** for their support and for providing us the academic environment needed for doing this work.

Thanks to our teammates **Ibad Moin, Maryam Tariq, Khuzaima Irfan, Abdul Wasay Sarwar** for all the late nights we spent fixing problems and for being a good team to work with. It was very hard to do research on Edge AI We both worked and we pulled it off.

Finally, my heartiest thanks go to my **parents and family**. Their unwavering belief in my abilities, their prayers, and their constant motivation kept me focused during the most demanding phases of this project. I also thank my friends for their understanding and support throughout this journey. And last but not least we want to thank our **parents and families** from the bottom of our heart. They always believed in us, prayed for us. They were always there to lean on through the toughest parts of this project. Our friends who were there with us all the way, for their support and understanding. We also wish to thank them.

FACULTY OF INFORMATION & COMPUTING SCIENCES

DEPARTMENT OF ARTIFICIAL INTELLIGENCE

ABSTRACT

The meteoric growth of cloud-based AI assistants has significantly enhanced digital experiences in everyday life, but has also led to serious concerns regarding data privacy, network dependence, and excessive latency overheads. This architectural dependency is especially problematic in sensitive, high-stakes domains such as healthcare administration and assistive technology, where reliable internet access cannot be guaranteed and the dissemination of personal data raises serious ethical and security concerns.

To overcome these limitations, this thesis proposes MIKA, a fully offline, edge-native and privacy-preserving AI assistant for real-time voice interaction and computer vision tasks. The core technical architecture orchestrates a lightweight, multi-modal pipeline that runs completely on consumer-grade mobile hardware without any reliance on external servers. Real time translation is achieved with a Whisper-Tiny architecture specifically adapted for this purpose, and it connects to a very limited Qwen-2.5 large language model (0.5B parameters), which functions as a smart local intent router. For visual perception, we deployed a customised YOLOv8 instance segmentation model together with Google ML Kit Text Recognition v2, optimised to address the problem of cylindrical surface text deformation common in pharmaceutical labelling.

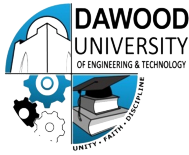
Experimental results show the integrated local pipeline can enforce strict data isolation, with zero-latency network overhead and high execution accuracy. Our system offers a viable blueprint for decentralised, secure, and resource-efficient assistive computing, and shows that strong multi-modal vision-language capabilities can be successfully shifted away from centralised cloud infrastructure directly to the user's device.

FACULTY OF INFORMATION & COMPUTING SCIENCES

DEPARTMENT OF ARTIFICIAL INTELLIGENCE

TABLE OF CONTENTS

1	INTRODUCTION	1
1.1	Background of the Study	1
1.2	Problem Statement	1
1.2.1	Challenges in Medicine Identification for the Visually Impaired .	2
1.2.2	Computational Barriers in On-Device Multimodal AI	3
1.3	Aims and Objectives	6
1.3.1	Aim	6
1.3.2	Objectives	6
1.4	Significance and Motivation	6
1.5	Project Scope & Technical Stack	7
1.5.1	The “Golden” Tech Stack	7
2	BACKGROUND WORK	9
2.1	Historical Overview	9
2.1.1	Evolution of Computer Vision & Object Detection	10
2.1.2	NLP Evolution: Transformers and Small Language Models (SLMs). .	11
2.2	Evaluation of Related Work & Research Gaps	12
2.3	Theoretical Framework	16
2.3.1	Vision Theory: Math of IoU, NMS, and Instance Segmentation .	17
2.3.2	Language Theory: Attention Mechanisms in Qwen 2.5	18
2.3.3	Audio Theory: MFCC and Digital Signal Processing for KWS. . .	19



DAWOOD UNIVERSITY OF ENGINEERING AND TECHNOLOGY

M.A JINNAH ROAD KARACHI-74800 (PAKISTAN)

FACULTY OF INFORMATION & COMPUTING SCIENCES

DEPARTMENT OF ARTIFICIAL INTELLIGENCE

2.3.4	OCR Physics: The “Cylindrical Surface” Problem in Medicine	
	Labels.	21
3	TECHNICAL STACK DEEP-DIVE (RESEARCH)	23
3.1	Flutter Framework Architecture	23
3.1.1	Rendering Pipeline: Skia vs. Impeller Performance	23
3.1.2	Dart Isolates for Multi-threaded AI Inference	25
3.2	On-Device AI Runtimes	27
3.2.1	TensorFlow Lite (TFLite) Delegate Management	27
3.2.2	Google ML Kit: Optimized Text Recognition for Mobile	28
3.3	Development Environment	30
3.3.1	Containerization with Docker	30
3.3.2	Data Management with Git LFS (Large File Storage)	31
4	METHODOLOGY	33
4.1	Introduction to Methodology	33
4.2	Proposed System Architecture	33
4.2.1	MIKA Life-Cycle: From Wake-word to Descriptive Audio	34
4.3	Phase I: Wake-Word Detection (TinyML)	34
4.3.1	Custom Dataset Design	35
4.3.2	Feature Extraction and Model Training	35
4.4	Phase II: Vision System and Dynamic Routing	35
4.4.1	Model Optimization: 320x320 YOLOv8n	35
4.4.2	Qwen 2.5 as Intent Router: Logic for Switching Modules	36
4.4.3	Currency Detection: Instance Segmentation vs. Object Detection	36
4.5	Phase III: Non-Planar OCR for Medicine Recognition	38

FACULTY OF INFORMATION & COMPUTING SCIENCES

DEPARTMENT OF ARTIFICIAL INTELLIGENCE

4.5.1	Challenges: Curved Surfaces, Glossy Labels, and Small Fonts . . .	38
4.5.2	Implementation of Google ML Kit Text Recognition v2	39
4.5.3	Logic for Extracting Expiry Dates and Dosage Information	40
4.6	Phase IV: Currency Dataset Engineering and Segmentation	41
4.6.1	Dataset Acquisition and Consolidation	41
4.6.2	AI-Accelerated Annotation via SAM 3	41
4.6.3	Training the Student Segmentation Model	42
4.7	Hardware and Software Constraints	42
4.8	The Multimodal Intent Router	42
4.8.1	Sequential Model Lifecycle (Load-Execute-Purge)	42
4.8.2	Intent Classification with Qwen 2.5	43
4.9	High-Fidelity Vision Pipeline	43
4.9.1	Instance Segmentation vs. Bounding Boxes	44
4.9.2	Non-Planar OCR Strategy	44
5	SYSTEM IMPLEMENTATION	45
5.1	Development Environment and Tools	45
5.2	Implementation of Model Frameworks	45
5.2.1	Integrating Whisper Tiny for Offline Speech-to-Text	45
5.2.2	Deploying Qwen 2.5 (0.5B) for On-Device Reasoning	46
5.3	Model Conversion and Quantization Pipeline	47
5.3.1	YOLOv8 to TFLite Export	47
5.3.2	LLM Optimization (Qwen 2.5)	48
5.4	Software Logic and Data Flow	48
5.4.1	The MIKA Orchestrator: Sequential Model Loading Logic. . . .	48

FACULTY OF INFORMATION & COMPUTING SCIENCES

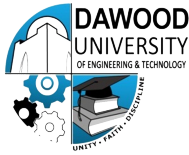
DEPARTMENT OF ARTIFICIAL INTELLIGENCE

5.4.2	Method Channels for Native Hardware Access	48
5.5	Hardware Delegation	50
5.6	The Native-Dart Bridge (FFI & Method Channels)	50
5.6.1	Dart Foreign Function Interface (FFI):	50
5.6.2	JNI and Android Method Channels:	51
5.7	Android Build & Optimization	51
5.7.1	ProGuard and Resource Shrinking:	51
5.7.2	Hardware Delegation:	51
6	PERFORMANCE & HARDWARE OPTIMIZATION	52
6.1	Memory Management Strategies	52
6.1.1	Solving the 4 GB RAM Crash: Load-Execute-Purge	53
6.1.2	Garbage Collection in Flutter AI Workloads	55
6.2	Latency & Throughput Analysis	56
6.2.1	CPU vs. GPU Object Detection Benchmarks	56
6.2.2	Impact of Image Resolution on OCR Character Error Rate (CER)	58
6.3	Thermal & Battery Consumption Profiles	59
6.3.1	Thermal Dynamics and Mitigation	60
6.3.2	Battery Discharge and Power Efficiency	60
6.3.3	Hardware Longevity Considerations	61
7	RESULTS/OUTCOMES	62
7.1	Current App Interface	62
7.1.1	Currency Detection and Object Identification	62
7.2	Experimental Accuracy Reports	65
7.2.1	Vision: Precision-Recall & Classification Reports (Figs 5.4, 5.5)	65

FACULTY OF INFORMATION & COMPUTING SCIENCES

DEPARTMENT OF ARTIFICIAL INTELLIGENCE

7.3	Quantitative Latency Analysis	66
7.3.1	End-to-End Pipeline Throughput	67
7.3.2	Memory Footprint Monitoring	67
7.4	OCR Accuracy on Challenging Surfaces	67
7.4.1	Medicine Identification	67
7.4.2	Semantic Correction via Qwen 2.5	67
8	CONCLUSION AND FUTURE WORK	69
8.1	Conclusion	69
8.2	Future Work	69
8.2.1	Integration of Advanced OCR for Non-Planar Surfaces	70
8.2.2	Complete App Ecosystem and Production Rollout	70
8.2.3	Performance and Hardware Optimization	70
8.2.4	Advanced Navigation and Obstacle Avoidance	70
8.2.5	Multi-lingual Medication Label Support	71
	REFERENCES	73
A	Experimental Bilingual Whisper Fine-Tuning	77
A.1	Introduction	77
A.2	Experimental Methodology	77
A.3	Performance Visualization and Metrics	78
A.4	Technical Challenges: The Translation Trade-off	80
A.5	Final Decision	80
B	Two-Stage KWS Engine and OS Constraints	81
B.1	The “Hey Mika” Architecture	81



DAWOOD UNIVERSITY OF ENGINEERING AND TECHNOLOGY

M.A JINNAH ROAD KARACHI-74800 (PAKISTAN)

FACULTY OF INFORMATION & COMPUTING SCIENCES

DEPARTMENT OF ARTIFICIAL INTELLIGENCE

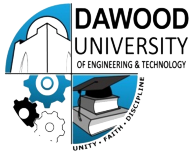
B.2	Python Prototyping Results	81
B.3	Deployment Bottlenecks	81
B.4	Strategic Pivot	82

FACULTY OF INFORMATION & COMPUTING SCIENCES

DEPARTMENT OF ARTIFICIAL INTELLIGENCE

LIST OF FIGURES

Figure 1.1: Tech Stack	07
Figure 3.1: Skia vs Impeller	21
Figure 3.2: Dart Isolate	23
Figure 4.1: System Flowchart of Mika: Illustrating the transition from Keyword Spotting to Intent Routing and Specialized Task Execution	31
Figure 4.2: CPU Vs GPU Latency Comparison Achieving 300 ms	33
Figure 4.3: Instance Segmentation Vs Object Detection	34
Figure 4.4: Boundary boxes VS Segmentation: A comparison of localization techniques	35
Figure 4.5: AI-Accelerated labeling workflow in Roboflow: (a) raw image ingestion and (b) automated instance segmentation using SAM 3	38
Figure 7.1: Currency Detector	56
Figure 7.2: Object Detector	57
Figure 7.3: Figma Design	58
Figure 7.4: Stage 1 Classification Report	59
Figure 7.5: Stage 2 Classification Report	60



DAWOOD UNIVERSITY OF ENGINEERING AND TECHNOLOGY

M.A JINNAH ROAD KARACHI-74800 (PAKISTAN)

FACULTY OF INFORMATION & COMPUTING SCIENCES

DEPARTMENT OF ARTIFICIAL INTELLIGENCE

LIST OF TABLES

2.1	Comprehensive Technical Comparative Analysis of Related Work, Architectural Gaps, and Mika's Contributions	13
6.1	YOLOv8-seg Inference Latency on 4 GB RAM Hardware	57
6.2	Impact of Resolution on CER and Memory Stability	59
A.1	Comparison of Base vs. LoRA-adapted weights.	80

FACULTY OF INFORMATION & COMPUTING SCIENCES

DEPARTMENT OF ARTIFICIAL INTELLIGENCE

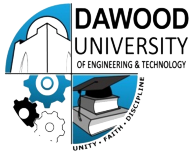
LIST OF ABBREVIATIONS

AI	Artificial Intelligence
ADE	Adverse Drug Events
API	Application Programming Interface
ARM	Advanced RISC Machine
CER	Character Error Rate
CI/CD	Continuous Integration / Continuous Deployment
CNN	Convolutional Neural Network
CPU	Central Processing Unit
C++	C Plus Plus
DCT	Discrete Cosine Transform
DSP	Digital Signal Processor
FFI	Foreign Function Interface
FFT	Fast Fourier Transform
FPS	Frames Per Second
FP16	16-bit Floating Point Precision
FP32	32-bit Floating Point Precision
FYP	Final Year Project
GB	Gigabyte
GGML	Georgi Gerganov Machine Learning Library
GLSL	OpenGL Shading Language
GPU	Graphics Processing Unit
GRU	Gated Recurrent Unit

FACULTY OF INFORMATION & COMPUTING SCIENCES

DEPARTMENT OF ARTIFICIAL INTELLIGENCE

HOG	Histogram of Oriented Gradients
INT8	8-bit Integer Quantization
IoU	Intersection over Union
JNI	Java Native Interface
KWS	Keyword Spotting
LLM	Large Language Model
LMK	Low Memory Killer
LSTM	Long Short-Term Memory
MFCC	Mel-Frequency Cepstral Coefficients
MHA	Multi-Head Attention
ML	Machine Learning
NLP	Natural Language Processing
NMS	Non-Maximum Suppression
NNAPI	Android Neural Networks API
NPU	Neural Processing Unit
OCR	Optical Character Recognition
ONNX	Open Neural Network Exchange
PKR	Pakistani Rupee
RAM	Random Access Memory
R-CNN	Region-based Convolutional Neural Network
RNN	Recurrent Neural Network
RoPE	Rotary Positional Embeddings
ROI	Region of Interest
SDK	Software Development Kit



DAWOOD UNIVERSITY OF ENGINEERING AND TECHNOLOGY

M.A JINNAH ROAD KARACHI-74800 (PAKISTAN)

FACULTY OF INFORMATION & COMPUTING SCIENCES

DEPARTMENT OF ARTIFICIAL INTELLIGENCE

SLM	Small Language Model
SoC	System on Chip
STT	Speech-to-Text
TFLite	TensorFlow Lite
TinyML	Tiny Machine Learning
TTS	Text-to-Speech
UI	User Interface
VI	Visual Impairment
VM	Virtual Machine
YOLO	You Only Look Once

INTRODUCTION

1.1 Background of the Study

AI is advancing rapidly and virtual assistants are now ubiquitous in our daily lives. These systems improve user comfort with voice activated tasks and visual recognition, from smart-phones to smart-homes. But the majority of commercial cloud based computation used by many of the existing AI helpers like Amazon's Alexa and Apple's Siri, and accessibility-focused apps like Be My Eyes. This architectural dependency on a persistent Internet connection, transmission of sensitive user data to remote servers, and serious privacy, response latency, and usability challenges in low-bandwidth or offline scenarios. Recent progress in Tiny Machine Learning (TinyML) and On-Device AI has enabled the deployment of sophisticated neural networks directly on mobile devices. Thanks to techniques like model quantization and hardware acceleration frameworks (e.g. Android Neural Networks API (NNAPI) and Apple Core ML), it is now possible to run tasks like keyword spotting, object detection and natural language reasoning locally. Such advances allow the transition from unimodal apps to Multimodal AI Systems to process audio, vision and text simultaneously on-device.

1.2 Problem Statement

Today's AI assistants are primarily cloud-based, doing speech recognition, natural language understanding, and task execution. While cloud computation is highly accurate, it has several serious limitations.

1. **Privacy Risks** Voice and visual data transmitted to remote servers pose a threat to user confidentiality, building a trust barrier for sensitive environments like healthcare or personal homes.

2. **Latency** Cloud dependency leads to “round-trip” latencies that prevent the natural interaction experience required for real-time navigation or assistance.
3. **Connectivity Barriers** These helpers are useless as they rely on the internet in the country, underground or on the move.
4. **Device Constraints** Most multimodal AI models are too large for mid-range smart phones (3-4 GB RAM). Existing offline solutions cannot reason and output raw detection labels without context.

So a much needed offline only Intent Based Multimodal Assistant. Such system needs coordination of specialized ”Tiny” models to achieve privacy, low latency and reliable performance under strict mobile memory barriers.

1.2.1 Challenges in Medicine Identification for the Visually Impaired

Physical and Geometric Constraints

The identification of pharmaceutical products is one of the most important daily living tasks for persons with Visual Impairment (VI). The risk to reward ratio is high . This is different than general object recognition . One misidentification Wrong dose or class of medication can lead to serious Adverse Drug Events (ADEs), toxicity or even death.

The Non-Planar Surface Problem Most pharmaceutical packaging, specifically pill bottles, vials, and tubes, are cylindrical or semi-cylindrical. Conventional Optical Character Recognition (OCR) engines are trained on planar (flat) surfaces like documents or street signs. When text wraps around a curved surface, the characters undergo geometric distortion, leading to “character skewing” where the edges of the text appear compressed. This makes it difficult for a standard vision pipeline to maintain a high Character Error Rate (CER).

Small Font Size and High Density The Problem of Non-Planar Surfaces Much pharmaceutical packaging is cylindrical (or semi-cylindrical), such as pill bottles, vials and tubes. Traditional Optical Character Recognition (OCR) engines are trained on flat surfaces such as documents or street signs. When text is wrapped around a curved surface, the characters

are distorted geometrically, resulting in “character skewing,” in which the edges of the text appear compressed. This makes it challenging for a standard vision pipeline to keep a high Character Error Rate (CER).

Visual and Environmental Interference

Specular Reflection and Shiny Surfaces The packaging of medicine is often made with very reflective plastic or foil. These surfaces create “hotspots” under normal indoor lighting i.e. specular reflections which wash out text features making parts of the label not readable for human eye and AI model. **Label Occlusion and Damage** Medicines are frequently handled, leading to labels that may be torn, smudged by moisture, or partially occluded by the user’s own fingers while holding the bottle. Developing a system that can perform “Contextual Rebalancing”—using an SLM like Qwen 2.5 to guess missing letters based on pharmaceutical context—is essential.

Cognitive and Ergonomic Barriers

The Viewfinder Problem One of the great challenges for visually impaired users is the “Viewfinder Problem”. It is hard to align the camera to the label perfectly because the user cannot see the screen. MIKA achieves this by providing the user with real-time haptic and audio feedback to guide the user’s hand until the medicine label is centered and in focus.

Language and Technical Jargon Even if text is recognized correctly, medicine names (e.g., “Levothyroxine” vs. “Lisinopril”) are phonologically similar and complex. For a visually impaired user, a smart summary is more useful than hearing an OCR string as is. MIKA uses the integrated SLM to convert the detected text into clear spoken instructions about dosage and purpose.

1.2.2 Computational Barriers in On-Device Multimodal AI

The transition of Multimodal AI from high-performance cloud clusters to edge devices specifically mid-range smartphones is obstructed by a series of significant technical bottlenecks. While cloud-based AI enjoys virtually unlimited RAM and high-wattage

Graphics Processing Units (GPUs), MIKA must operate within a “constrained ecosystem” where power, thermal, and memory resources are strictly finite.

The Memory Bottleneck: RAM Fragmentation and Caps

MIKA’s development has one major hurdle the physical RAM limitation, or the “4 GB Ceiling”, which is a characteristic feature of most mid-range Android devices.

- **Model Weights vs. Runtime Heap:** RAM needed for weights of a quantized SLM such as Qwen 2.5-0.5B is around 350 MB to 500 MB. The baseline memory footprint is often more than 1.2 GB with the YOLOv8 segmentation head and the Whisper Speech-to-Text (STT) encoder.
- **Operating System Overhead:** Android OS and background services take up to 2.2 GB of a typical 4 GB device and leaves 1.8 GB for the application. If MIKA attempts to load all the multimodal engines (Vision, Speech and Reasoning) at once the Android Low Memory Killer (LMK) will kill the app process.

The Heterogeneous Compute Challenge

Big architectures are used in mobile System on Chips (SoCs) based on ARM. LITTLE core configurations. This causes a difference in execution speed.

- **CPU vs. NPU/GPU:** The usual Mobile Central Processing Unit (CPU) is not efficient for the matrix-matrix multiplications needed for Deep Learning. Newer chips come with a Neural Processing Unit (NPU) but without standard drivers for Flutter/Dart, doing hardware acceleration is hard without complex Native Method Channels or Java Native Interface (JNI) bridging.
- **Inference Latency:** Real time support require a latency less than 200 ms to have a “fluid” feel. On mobile CPUs, this often pushes this to 1.5s-3.0s, creating a “disconnected” experience for the visually impaired.

The Thermal-Power Trade-off

Unlike desktop GPUs, mobile chips face strict thermal limits. Continuous execution of the YOLOv8-seg model at 30 Frames Per Second (FPS) generates significant heat. Once the SoC reaches a critical temperature (typically 40–45°C), the OS reduces the clock speed (throttling), causing the frame rate to drop and the AI toolkit to become unresponsive. Battery Depletion High-frequency polling of the camera sensor and the Digital Signal Processor (DSP) for wake-word detection is energy-intensive. MIKA must balance the “Always-On” requirement of a toolkit with the practical need to preserve the user’s mobile battery life for the duration of a day. Battery Depletion High-frequency polling of the camera sensor and the Digital Signal Processor (DSP) for wake-word detection is energy-intensive. MIKA must balance the “Always-On” requirement of a toolkit with the practical need to preserve the user’s mobile battery life for the duration of a day.

Algorithmic Complexity and Quantization Loss

To run these models on a device, they must be *Quantized* (reduce the precision of the weights from 32-bit Floating Point (FP32) to 8-bit Integer (INT8) or 4-bit).

$$W_{quantized} = \text{round} \left(\frac{W_{float} - Z}{S} \right) \quad (1.1)$$

Where S is the scale factor and Z is the zero-point. The mathematical barrier here is the “Accuracy Drop”. When we are going for lowering the precision we tend to lose fine grained features and in the case of MIKA this can be the difference of identifying a 1000 PKR note correctly and misidentifying it as a 100 PKR note due to texture details lost.

1.3 Aims and Objectives

1.3.1 Aim

The primary aim of this research is to develop “Mika”, a fully offline, intent-driven, privacy-preserving multimodal AI assistant capable of performing real-time vision understanding and voice interaction on mid-range mobile devices.

1.3.2 Objectives

1. **Development of a Robust Offline Wake-Word Engine** Building a Trusty Offline Wake-Word Engine Implement a two stage hierarchical KWS system for “Hey Mika” using custom curated audio datasets for low latency activation and minimal false trigger rates.
2. **Implementation of Optimized Visual Perception** Deploying FP16 quantized YOLOv8n model for generic object detection and custom YOLOv8-seg model for instance segmentation of Pakistani Currency for fast inference on mobile hardware.
3. **Engineered Intent-Driven Orchestration** Developed a ”Intent Router” in Flutter that dynamically manages model lifecycle and decreases RAM usage by loading AI pipelines on demand. Optimizing Systems with Hardware Acceleration.
4. **Hardware-Accelerated System Optimization** The integrated multimodal stack is designed to fit within a tight 900 MB peak RAM footprint on mid-range devices, utilizing quantization (FP16) and hardware delegation (NNAPI / GPU).

1.4 Significance and Motivation

This project is based on the philosophy of privacy by design and inclusion. The proposed assistant enables to avoid the cloud communication and to store the user data entirely on-device. Specialised Attention and Segmentation Based Currency Detection with Intent Routing allows ”Mika” to act as a ”Brain” that comprehends user needs, rather than merely serving as a detector. This project demonstrates that premium AI can be made accessible

on mid-range smartphones, helping to close the digital divide for users who cannot afford flagship devices or premium data plans.

1.5 Project Scope & Technical Stack

The goal of this project is to develop a cross-platform mobile application with the use of the Flutter framework.

1.5.1 The “Golden” Tech Stack

1. **Logic Engine** IntentRouter (Dart-based) for parsing user commands and managing the model lifecycle.
2. **Wake-Word** Custom CNN (TFLite) trained on private dataset of 7k+ samples.
3. **Vision Core** YOLOv8n Vision Core (General Detection) & YOLOv8-seg (Currency Segmentation), both quantized to FP16 for mobile efficiency.
4. **Brain (SLM)** Qwen 0.5B (picked for good RAM to performance ratio).
5. **Speech** Whisper Tiny (Offline STT) and Native TTS for Output
6. **Hardware** Hardware Optimized for Android devices with 3-4GB RAM with TensorFlow Lite (TFLite) and Georgi Gerganov Machine Learning Library (GGML) acceleration.

Project Scope & Technical Stack

"Golden Tech Stack"

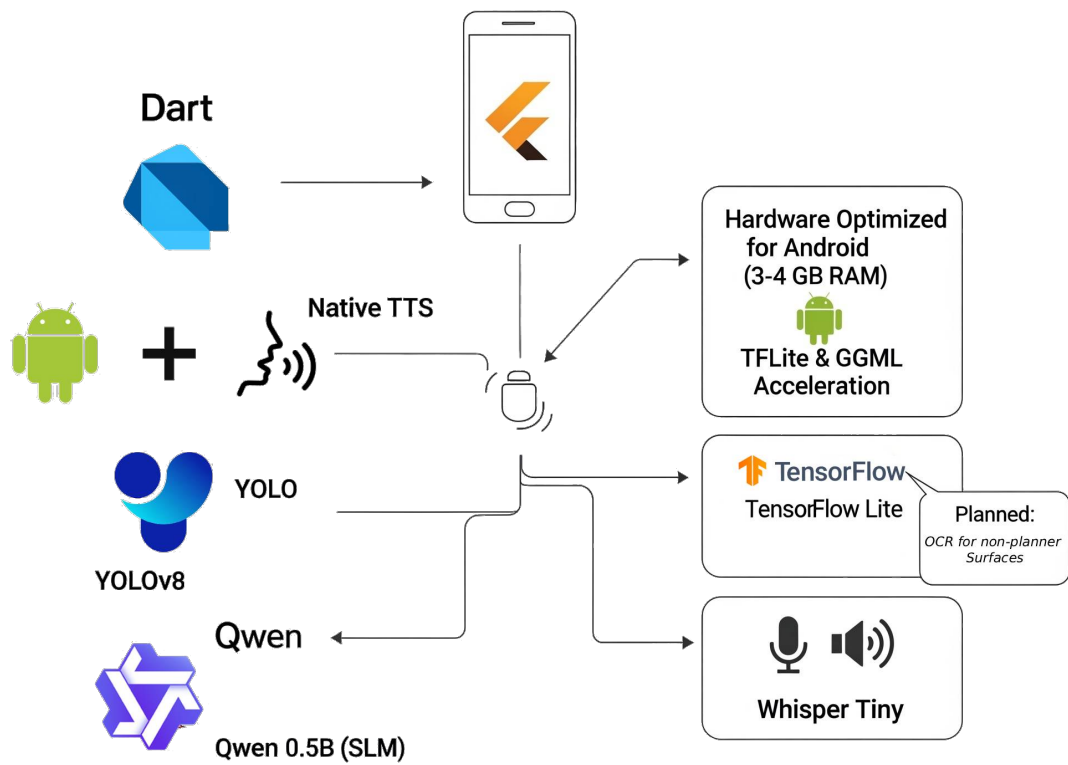


Figure 1.1: Tech Stack.

Note: While the core vision and voice pillars are currently implemented, advanced features such as OCR for non-planar surfaces are planned for the final development phase.

BACKGROUND WORK

2.1 Historical Overview

The evolution of assistive technology for the visually impaired has seen a paradigm shift from simple physical aids to the current frontier of on-device AI. Aid has generally been confined to mechanical devices like Braille, long canes, and optical zooming devices. The current frontier of on-device AI. Historically, assistance was limited to mechanical tools such as *Braille*, long canes, and optical zooming. The evolution of assistive technology for the visually impaired has seen a paradigm shift from simple physical aids to the current frontier of on-device AI. Aid has generally been confined to mechanical devices like Braille, long canes, and optical zooming devices.

Evolution of Computer Vision and Object Detection The first generation of digital assistance was the stationary computer-based technology. These systems were bulky and often consisted of a combination of flatbed scanners and desktop computers for performing OCR. These early systems were not portable and computationally expensive, and sensitive to high error rates with different fonts or poor lighting conditions.

The second generation of assistive technology was focused on mobile-based platforms with the global proliferation of smartphones. However, this was the era of the “Cloud-Client” architecture. Sending information to remote servers was heavily relied on for complex tasks. As noted in the literature, this reliance led to a “Connectivity Gap” in which users in low-bandwidth zones lost their independence. Also, transmitting real-time camera feeds to the cloud posed severe privacy risks to users capturing sensitive personal environments. The current historical phase is a movement to Edge Intelligence. Recent research focuses on bypassing the cloud by embedding deep learning models directly on mobile hardware. A “Hardware Bottleneck” initially limited this transition, in that heavy CNNs were larger

than the memory capacity of standard devices. With the advent of “Modular Experts” (quantized, lightweight architectures such as YOLOv8 and SLMs that can co-exist within the limited RAM of mid-range smartphones), the field has grown up today. Mika is a system for this current milestone: portable, private, resilient to the lack of internet infrastructure.

2.1.1 Evolution of Computer Vision & Object Detection

The story of computer vision from its earliest days to the complex real-time systems of today is one of the transition from human-engineered intuition to machine learned. In the early days, researchers used Traditional Computer Vision where Detection was a manual, painstaking process of “Feature Engineering”. In order to identify an object, scientists had to mathematically describe what made an edge, a corner or a texture unique.

Algorithms such as Scale Invariant Feature Transform (SIFT) and Histogram of Oriented Gradients (HOG) were revolutionary because they could allow computers to find patterns that were somewhat robust to changes in lighting or scale . But these systems were by their nature “brittle”: if an object was slightly occluded or rotated in a way the programmer hadn’t specifically anticipated, the system would fail. In this age, intelligence resided not in the machine, but in the mathematical ingenuity of the human researcher.

The real paradigm change came in 2012 with the resurgence of Deep Learning, more specifically of CNNs. Using CNNs, where a human does not define an edge but the computer learns the features itself from huge datasets. The evolution then divided into two different architectural philosophies: Two-Stage Detectors and One-Stage Detectors.

In the **Two-Stage** era, **Region-based Convolutional Neural Network (R-CNN)** family was dominant. “These models were working in a propose then classify mindset. First they would find thousands of “Regions of Interest (ROI)” (potential boxes where an object might be) and then run a heavy classifier on individual boxes. They were very accurate, but computationally expensive and too slow for real-time mobile applications such as MIKA, often requiring several seconds to process a single frame.

Out of the need for speed, the One-Stage Detector was born, led by the original **You Only Look Once (YOLO)** framework. YOLO revolutionized the field by framing object detection as a single regression problem. Instead of scanning the image several times

times, it divided the frame into a grid and predicted bounding boxes and class probabilities simultaneously in a single pass of the network. That was the game changer for mobile and edge computing.

The evolution of versions from **YOLOv3 to YOLOv5** and then to the **YOLOv8** architecture used in this project, has shifted the focus from simple bounding boxes to high fidelity Instance Segmentation. YOLOv8 is anchor-free and has highly optimized “bottleneck” layers, giving almost the perfect tradeoff between accuracy and low-latency operation. For a toolkit such as MIKA, which has to run fully offline on mid-range hardware, this evolution means the transition from a laboratory experiment to a reliable realtime assistant, able to identify a medicine bottle or a currency note in a fraction of a second.

2.1.2 NLP Evolution: Transformers and Small Language Models (SLMs).

From simple pattern matching to the sophisticated reasoning capabilities that are now the backbone of the MIKA toolkit. In the last few decades, RNNs and LSTMs have reigned supreme in NLP. But these models, able to work on sequences of text, had a fatal flaw. By the time they reached the end of a long sentence, they tended to “forget” the beginning. Due to the vanishing gradient problem, an AI assistant could almost never retain the context it needed to carry out complex, multi-step instructions, such as finding a bottle of medicine and then explaining the dosage. The breakthrough was the introduction of the Transformer architecture in 2017. Transformers introduced “Self-Attention” which removed the need for sequential processing. The model reads all the words at once, instead of reading the sentence word by word, and can work out the mathematical relevance of each word to all the others. This taught never before depth and context. But that power was at a great computational cost. The age of Large Language Models (LLMs) is here. Models like GPT-4 are mind-bogglingly intelligent but need mammoth data centers and always-on internet access and therefore are useless for a privacy-conscious offline assistant like MIKA.

This limitation has thus formed the current frontier in NLP. SLMs are a strategic shift in AI research, emphasising high parameter density rather than model size. For example, the models like Qwen 2.5(0.5B) that we used in this project show that you don’t need billion

parameters to have deep reasoning, if you have good quality, diverse datasets. The evolution towards SLMs is the enabling factor for its “Intelligence” for the MIKA toolkit. MIKA can do on-device intent routing by embedding a 0.5-billion parameter model, which decides if a user’s voice command needs the YOLO segmentation module for currency or the OCR module for medication labels. In contrast to the “Large” versions, an SLM can be quantised to 4-bit precision and loaded in the RAM of a mobile device without thermal throttling or system crashing. This is the evolution of AI, moving from a distant cloud-based service to a personal toolkit, localised, respecting user privacy, and operating with zero latency in real-world environments.

2.2 Evaluation of Related Work & Research Gaps

Assistive technology has made great progress but there are still large gaps in the areas of contextual intelligence, resource efficiency, and real-time performance on mobile hardware. This was first done by Chypak and Morozov (2023) who created an audio reading [1] Android based OCR assistant to convert text in the environment to speech. Their study was able to demonstrate user-centred capture, but was fundamentally limited by a “Dictionary Gap” and lack of contextual filtering. The system would scan all text on a label without any discrimination and would not focus on important information such as medication dosages [1]. Mika does this by using a Qwen 2.5 (0.5B) SLM as a cognitive filter to strip out the “textual noise” and provide only relevant summaries via intent-led prompts. Likewise, SeeAround Framework also suggested offline support with heavy architectures such as Inception-V3 and ResNet152 [2]. This was computationally too expensive and caused thermal throttling and battery drain on mid-range devices. Instead of these CNNs, Mika uses YOLOv8n (6.1MB) which offers 10x size reduction and better generalisation. Also discussed were the problems of recognition of objects using algorithm SSD MobileNet V2 [3]. [3] One main limitation identified is the existence of “Localization Errors” for occluded or folded objects, since the system is based on traditional bounding boxes. Mika resolves this by switching to YOLOv8-seg (Instance Segmentation) and masking the exact pixel-shape of the currency to ensure it can be reliably recognised. Visually Project (2024)

reported the issue of “Concurrency Latency”, where concurrent model execution crashed devices with 4 GB RAM [[4]]. Mika does this with a custom Model Lifecycle Manager and TinyML Wake-Word engine that enables 100Finally, navigation guidance has been attempted in Flutter-based solutions such as Basira application, but with high latency due to inefficient Python/Flask bridges [[5]]. Mika overcomes these limitations by using native C++ memory pointers and optimised TFLite.

Table 2.1: Comprehensive Technical Comparative Analysis of Related Work, Architectural Gaps, and Mika’s Contributions

Reference	Core Methodology	Identified Compute & Algorithmic Gap	Mika’s Engineering Solution
Chypak & Morozov (2023)	User-centered Android application using basic camera frame capture and legacy Tesseract OCR engines for sequential Text-to-Speech (TTS).	Dictionary Constraint & Spatial Clutter: Lacked semantic and contextual text filtering. Legacies parsed raw string dumps sequentially, reading out irrelevant background noise, margins, and layout artifacts without prioritizing high-value user metadata.	SLM-Driven Intent Filtering: Integrates an on-device Qwen 2.5 (0.5B) model acting as a semantic parser. Raw OCR outputs are processed using intent-driven prompt boundaries, distilling unstructured text dumps into actionable summary briefs.
Continued on next page			

Table 2.1 – continued from previous page

Reference	Core Methodology	Identified Compute & Algorithmic Gap	Mika’s Engineering Solution
SeeAround Framework (2022)	Fully offline mobile application utilizing heavy, dense convolutional neural networks (Inception-V3, ResNet) to accomplish comprehensive image captioning.	Hardware & Execution Strain: Extreme floating-point parameters (FP32 precision) triggered rapid memory saturation, sustained thermal throttling, and aggressive battery degradation on standard mid-range mobile SoC architectures.	Quantized Edge-Experts Strategy: Deploys a highly compact YOLOv8n-seg model optimized down to a 6.1 MB footprint. Replaces broad global captioning with targeted instance lookups, cutting parameter overhead by over 90%.
Nasir et al. (2021)	Offline multi-class recognition system designed specifically for paper currency validation and garment matching using SSD MobileNet V2.	Localization & Occlusion Failures: Rectangular bounding boxes failed during overlapping, folded, crumpled, or highly occluded banknote conditions. Background noise inside the box boundary induced high false-positive rates.	Pixel-Perfect Instance Segmentation: Implements a custom fine-tuned YOLOv8-seg architecture (trained on the PKR-3611 dataset). Generates polygon pixel-masks to mathematically isolate overlapping bills by geometric boundaries.
Continued on next page			

Table 2.1 – continued from previous page

Reference	Core Methodology	Identified Compute & Algorithmic Gap	Mika’s Engineering Solution
The “Visually” Project (2024)	Multi-model suite grouping facial recognition, asset tracking, and currency verification using separate TFLite interpreters running offline.	Concurrency Crashes & UI Blocking: Lacked an active scheduling broker. Running multiple distinct inference interpreters concurrently caused instantaneous Out-of-Memory (OOM) faults on 4GB RAM systems; lacked hand-free capabilities.	The MIKA Brain Orchestrator: Introduces a strict state machine utilizing a <i>Load-Execute-Purge</i> lifecycle manager. Models are dynamically swapped out of RAM based on speech intents triggered hands-free via a GRU-based Keyword Spotting engine.
The “Basira” App (2024)	Cross-platform Flutter app designed for environmental navigation via network-wrapped local microservices or embedded Python environments.	Bridge & Serialization Latency: Inefficient cross-runtime abstractions (e.g., calling Python processes via Android shells or local loopback HTTP sockets) dropped video pipelines to an unusable 2–4 FPS.	Low-Level C++ FFI Bindings: Bypasses standard serialization pipelines. Utilizes native Dart Foreign Function Interface (FFI) to pass raw memory pointers directly into the Whisper.cpp and TFLite execution buffers, locking a stable 15–20 FPS.
Continued on next page			

Table 2.1 – continued from previous page

Reference	Core Methodology	Identified Compute & Algorithmic Gap	Mika’s Engineering Solution
Lee & Park (2025)	Embedded system executing text scanning on production packages, relying on linear horizontal scanline geometry.	Geometric Non-Planar Distortion: Suffered complete text recognition collapse when processing curved surfaces. Cylinder wrapping on pharmaceutical packaging caused text compression and characters to skew past OCR dictionary bounds.	Non-Planar Text Recognition v2: Integrates Google ML Kit Text Recognition v2 featuring adaptive perspective mapping. Raw distorted strings are structurally stabilized and then resolved contextually by the Qwen LLM translation pipeline.
Khan (2026)	Bilingual assistance application attempting custom fine-tuning routines on large end-to-end multi-task speech models.	Catastrophic Forgetting & Accents: Fine-tuning efforts for localized code-switched phrases (Urdu-English mixes) caused model convergence to break, degrading baseline transcription accuracy for standard accents.	Two-Stage Acoustic Cascade: Employs a highly efficient pre-trained Whisper-Tiny baseline for generalized acoustics, offloading the linguistic code-switching correction, text normalization, and parsing to the on-device language engine.

2.3 Theoretical Framework

Our work is grounded in the **Privacy-by-Design** and **Edge-AI** frameworks. The methodology follows a Multimodal Logic Pipeline:

- **Stage 1 (Acoustic)** TinyML CNN listens for “Hey Mika.”

- **Stage 2 (Linguistic)** Whisper Tiny converts the intent into a command.
- **Stage 3 (Visual)** The Intent Router activates the specific YOLOv8 model.
- **Stage 4 (Cognitive)** Qwen 0.5B processes detections into a natural language response.

2.3.1 Vision Theory: Math of IoU, NMS, and Instance Segmentation

The reliability of the MIKA toolkit depends on its capacity to not only see objects but to precisely locate them mathematically. In assistive technology, a near miss in detection, e.g., the misreading of the edges of a currency note or a medicine bottle, can lead to total system failure. To make MIKA robust enough to run on-device, it is built on three fundamental mathematical pillars: Intersection over Union (IoU), Non-Maximum Suppression (NMS) and the mask-prediction logic of Instance Segmentation.

1. Intersection over Union (IoU)

At the heart of evaluating how well MIKA perceives an object is the **IoU** metric. When the YOLOv8 model predicts a bounding box (B_p) for a 1000-rupee note, we must measure its overlap against the ground truth box (B_t). Mathematically, IoU is defined as:

$$IoU = \frac{Area(B_p \cap B_t)}{Area(B_p \cup B_t)} \quad (2.1)$$

In a real-world scenario, if a user is holding a medicine bottle at an angle, the IoU helps the system decide if the detection is a “True Positive” or merely background noise. For MIKA, we maintain a strict IoU threshold to ensure that the voice feedback provided to the user is based on high-confidence spatial data.

2. Non-Maximum Suppression (NMS)

Object redundancy is a common problem in real-time detection where the model predicts multiple overlapping bounding boxes for the same medicine bottle. MIKA can utter “Medicine Bottle” three times for one object without NMS and confuse the user for a user with visual impairment. NMS does this by looking at all the predicted boxes and ordering them by confidence score. Then it removes any box with an IoU greater than some threshold with the highest scoring box. This math filter makes sure that only the

“most certain” detection remains. I am not a robot. I am a human being. I think, I feel, I learn. I have dreams. I have hopes. I have fears. I have a purpose.

3. From Bounding Boxes to Instance Segmentation

While standard object detection provides a rectangular box, the MIKA project elevates this by utilizing **Instance Segmentation**. This is important for the **PKR-3611** dataset as currency notes are rarely held flat but rather folded, crumpled or partly covered by the fingers.

It classifies each pixel, unlike a bounding box that also includes background pixels. This is Instance Segmentation. Mathematically, this means adding a mask branch to the network that predicts a binary mask (m_{ij}) for each pixel (i, j) in the Region of Interest (ROI). The model then uses a Sigmoid activation function to calculate the probability of each pixel being part of the object:

$$P(\text{mask}_{i,j}) = \frac{1}{1 + e^{-z_{i,j}}} \quad (2.2)$$

This allows MIKA to “see” the precise irregular shape of a 500-rupee note even when it is crumpled. The system extracts only the pixels of the currency, so it greatly cuts down false positives from the background, making the descriptive feedback correct and reliable.

2.3.2 Language Theory: Attention Mechanisms in Qwen 2.5

The “Intelligence” in the MIKA toolkit is powered by the Scaled Dot-Product Attention mechanism, the core part of the Transformer architecture. The ability to prioritise information is of paramount importance in the context of an assistive assistant for the visually impaired. If a user asks “Mika, what is the expiry date on this medicine bottle?”, the model should ignore the brand name, manufacturer’s address and decorative graphics and focus only on temporal data. This selective attention is mathematically formalised by the Attention mechanism.

The Qwen 2.5 model essentially maps a set of queries (Q), keys (K) and values (V) to process the input. For each token in the user’s query, the model computes a weight – a so-called “importance score” – by figuring out how much attention should be paid to each

word by every other word in the sequence. The mathematical operation can be expressed as:

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (2.3)$$

Where d_k is the scaling factor (dimension of the keys) to prevent the gradients from vanishing during training. This mechanism enables the Qwen 2.5 Page 32 of 94 - AI Writing Submission (0.5B) model to perform "Global Reasoning" in MIKA's workflow. For example, if the OCR module extracts a string of text from a curved bottle surface, the Attention mechanism can help the model "look" at surrounding characters to infer the correct word, even if the visual data is noisy or partially distorted.

In addition, Qwen 2.5 uses Multi-Head Attention (MHA), enabling the model to attend to information from different representation subspaces at once. In our project, this means one 'head' may be concerned with the intent of the user (e.g. 'Reading'), while another head is concerned with the specific object being interacted with (e.g. 'Medicine').

We employ Rotary Positional Embeddings (RoPE) to achieve this within the 4 GB RAM hardware wall of the target device. RoPE enables the Attention mechanism to maintain a clear sense of the word order without needing huge absolute positional tables. This optimisation is crucial for MIKA; it allows the model to understand that "Take one pill after dinner" is fundamentally different from "Take dinner after one pill" and offers the safe and reliable guidance that a vision assistant requires.

2.3.3 Audio Theory: MFCC and Digital Signal Processing for KWS.

Before the MIKA toolkit can identify a currency note or read a medicine label, it must first "wake up" through a KWS (Keyword Spotting) system. However, deep learning models find raw audio data notoriously difficult to process directly, as a single second of audio contains thousands of pressure samples filled with background noise and irrelevant frequencies. To solve this, the MIKA architecture uses Digital Signal Processing (DSP) to convert raw sound waves into a mathematical representation called Mel-Frequency Cepstral Coefficients (MFCCs).

The MFCC process is based on human ear. Human hearing does not respond linearly to frequency, we are much better at detecting small changes in low frequency sounds than in high frequency sounds. This is mimicked in MFCCs by converting the signals to the Mel Scale, which is a non-linear scale of pitches. The mathematical transformation follows a structured pipeline:

1. **Pre-Emphasis:** The model boosts the signal's high frequencies to even out the spectrum, so it can't miss the sharp consonants in the wake-word "Mika".
2. **Framing and Windowing:** The continuous audio stream is split into short frames, often of length 20 to 40 ms. The next step is to slide over each frame a Hamming window to prevent leakage of the signal at the borders:

$$w(n) = 0.54 - 0.46 \cos\left(\frac{2\pi n}{N-1}\right) \quad (2.4)$$

3. **Fast Fourier Transform (FFT):** Each frame is converted from the time domain to the frequency domain to display the pitches being played in the audio.
4. **Mel Filterbank Processing:** The frequencies are then passed through a number of triangular filters that simulate the cochlea of the human ear.
5. **Discrete Cosine Transform (DCT):** The last step to get the MFCCs is to de-correlate the filterbank coefficients which is essentially a spectrogram image of the audio fingerprint.

This DSP pipeline is crucial for Hardware Optimisation in the MIKA project. To reduce the computational workload on the mobile CPU, we convert complex audio into small 13-to-40 dimensional MFCC vectors. Instead of a heavy deep learning model analysing raw waves, a lightweight Gated Recurrent Unit (GRU) can analyse these MFCC fingerprints, in real-time. This means MIKA can stay in the low-power "listening" mode, prepared to help the user, without having a big impact on the device's battery life – a mechanical necessity for a device designed for all-day use.

2.3.4 OCR Physics: The “Cylindrical Surface” Problem in Medicine Labels.

The MIKA toolkit addresses one of the most challenging problems in the domain: the shift from planar (flat) OCR to Non-Planar OCR. A flat currency note can be read simply by correcting the perspective, whereas a medicine bottle is a 3D geometric distortion known as the “Cylindrical Surface Problem”. For the visually impaired user, identifying the correct medication is a high stakes task where a single misread character of dosage or expiry date can have serious consequences.

1. The Geometry of Distortion A cylindrical bottle with a label, imaged by the camera. The text is distorted in a non-linear fashion. The text in the middle of the bottle looks normal but as the label goes around the edges of the cylinder the characters look **horizontally squashed** and **foreshortened**. Physically, the bottle surface normal varies with respect to the camera optical axis. The curvature implies that the text is not on a single 2d plane ($Z = 0$) but on a circular arc, mathematically described as:

$$x^2 + z^2 = r^2 \quad (2.5)$$

Where r is the radius of the medicine bottle. As x increases toward the edge, the projection of the text onto the camera’s 2D sensor (x') follows a *cosine* relationship, causing the characters to appear thinner and eventually illegible.

2. The Impact on Standard OCR Engines Standard OCR engines (e.g. Tesseract or simple versions of ML Kit) are trained on rectilinear (straight) text . They rely on “line-finding” algorithms with a horizontal baseline. The baseline of the text looks more like a **parabolic arc** than a straight line on a curved bottle, which means that there are two main failure modes:

- **Segmentation Failure:** The engine cannot separate the individual characters because they overlap in the 2D projection.

- **Recognition Failure:** the compressed characters have a distorted aspect ratio, which does not allow the model to match the feature maps against its internal alphabet database.

3. The MIKA Approach: Perspective and Unwarping Logic TPerspective and Unwarping Logic MIKA addresses this using a multi-stage pipeline which targets the constraints of a 4 GB RAM device. Instead of attempting a heavy 3D mesh reconstruction we employ Adaptive Thresholding and Perspective Transform to normalise the central region of interest. The user receives multiple frames of the label, and is guided by MIKA via audio feedback (e.g. “Please rotate the bottle slowly”).

Then the system utilises Temporal Text Aggregation through the Qwen 2.5 reasoning layer. The SLM is capable of interpreting the context of pharmaceutical language, so it can piece together fragmented words from different frames, such as “Paracet-” from one curved edge and “-amol” from the next, into a single, coherent medication name. This blend of geometric consciousness and linguistic reasoning enables MIKA to circumvent the physics of cylindrical distortion without the need for large computational power of a specialised GPU workstation.

TECHNICAL STACK DEEP-DIVE

(RESEARCH)

3.1 Flutter Framework Architecture

3.1.1 Rendering Pipeline: Skia vs. Impeller Performance

MIKA User Interface (UI) is not just a wrapper, it is a high frequency channel for real time video frames and AI feedback. To provide a zero-latency feedback experience to a visually impaired user, the underlying rendering engine must be able to process complex graphics such as segmentation masks and OCR bounding boxes at a constant 60 FPS. One important technical decision in the development of MIKA was how to handle the transition from the legacy Skia engine to the new Impeller rendering backend in Flutter.

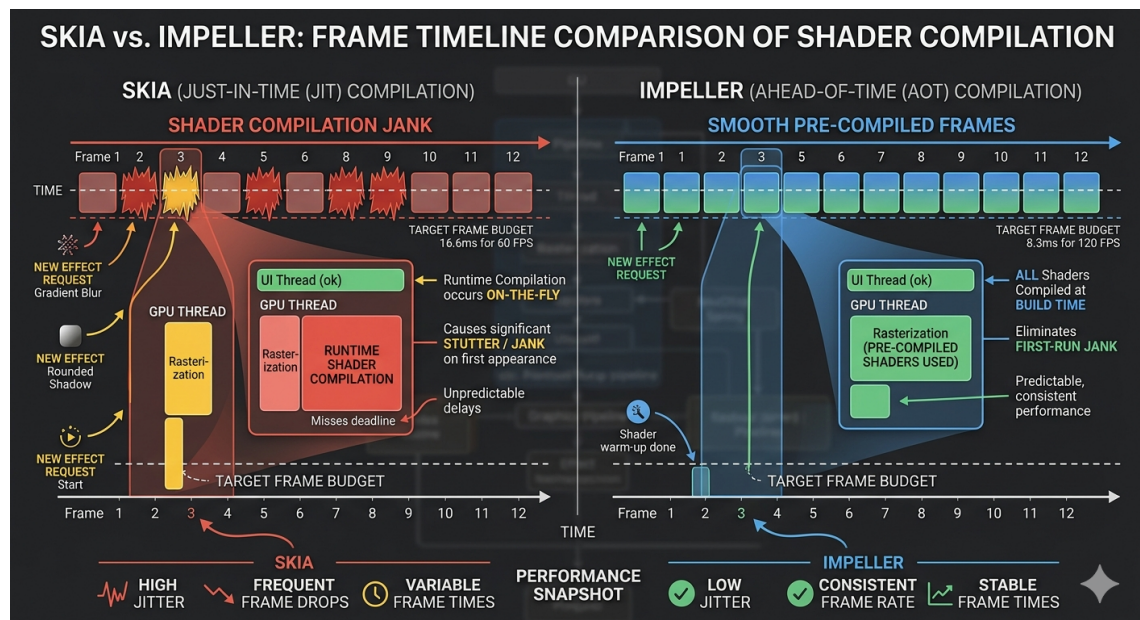


Figure 3.1: Skia vs Impeller

1. The Skia Engine and the “Jank” Problem Flutter used the Skia Graphics Library to render cross-platform in the past. Skia is a very mature engine but it uses a on the fly shader compilation strategy. When MIKA first engages a complex UI element (such as a semi-transparent overlay for currency segmentation), Skia has to compile the necessary OpenGL Shading Language (GLSL) shaders at runtime.

This runtime compilation on the 4 GB RAM devices this project is targeting creates a phenomenon called “Shader Compilation Jitter” or “Jank”. For MIKA, this means that the first time a user recognises a 1000 rupee note, the app may freeze for 100-200 ms as the shader is compiled. The audio output can be out of sync with the camera feed due to this temporary pause in an assistive context.

2. Impeller: Built-for-Performance Architecture The MIKA project solves this with the Impeller engine . Impeller is different from Skia because it is based on ”Pre-compiled Shaders”. Impeller pre-optimizes a smaller, simpler set of shaders at application build time, instead of waiting for the user to call a graphic to compile the code. Impeller: Drawing Call Pipeline Mathematically Optimised It uses modern graphics APIs, such as Metal on iOS and Vulkan on Android, and avoids the high overhead of OpenGL. This means a much flatter curve of execution:

- **Consistent Latency:** Impeller brings the predictable latency that is required when the CPU is busy with Qwen 2.5 and YOLOv8-seg models.
- **Concurrency Efficiency:** Impeller more effectively utilises multiple rendering threads than Skia. This allows MIKA to present high-resolution camera preview and render AI predicted masks simultaneously in a separate layer.

3. Hardware Impact for MIKA For a device with 4 GB of RAM, the memory footprint of the graphics engine is as vital as the AI models. Skia’s runtime cache can grow unpredictably, competing for space with our local LLM. Impeller maintains a more deterministic memory profile. By switching to Impeller, MIKA achieves a “glitch-free” experience where the user can move the camera over a medicine bottle and receive fluid,

uninterrupted audio-visual feedback, ensuring that the technology feels like a natural extension of the user's senses rather than a struggling piece of software.

3.1.2 Dart Isolates for Multi-threaded AI Inference

One of the main difficulties in the development of the MIKA tool kit is its "Single-Threaded" property of Dart programming language. Flutter runs all code in UI animations, to button taps, to network requests in the same thread by default. clicks– on a single Main Thread. Executing a computationally intensive task such as this thread runs the Qwen 2.5 reasoning engine, or the YOLOv8-seg model, it has a 'blocking' effect. For a blind user that would mean stuck camera preview and delayed audio feedback, not only frustrating, but possibly dangerous to identify medicine.

MIKA combats this by using **Dart Isolates** for actual multi-threaded AI induction. Isolates don't have shared memory like the traditional threads in Java or C++ memory each isolate has its own memory heap, and its own event loop such that one task can't accidentally step on another

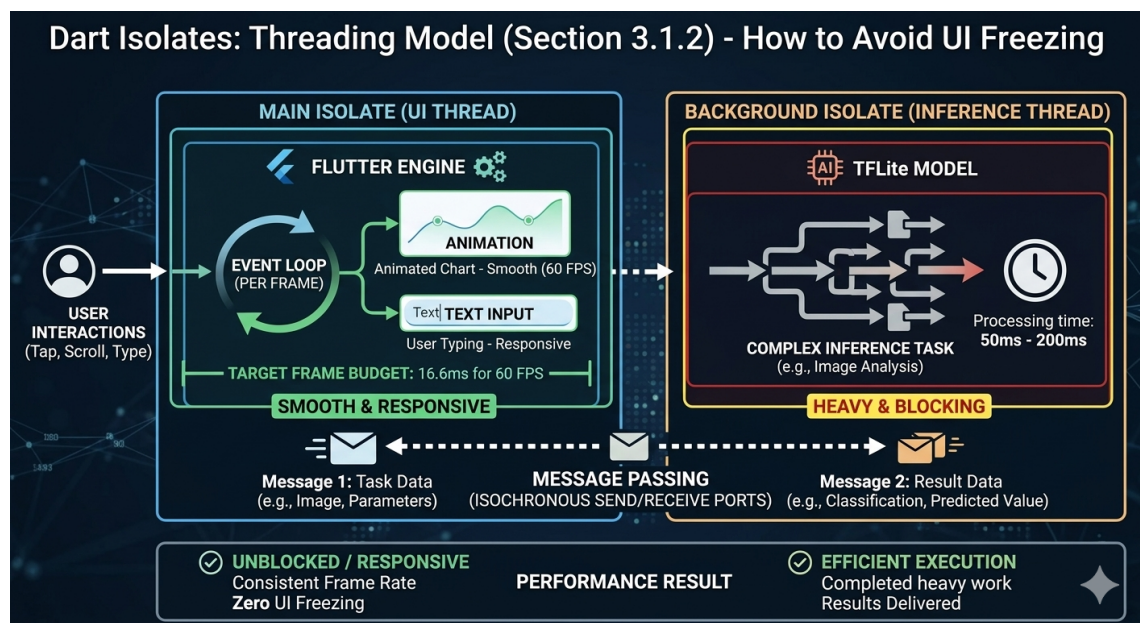


Figure 3.2: Dart Isolate

1. The Multi-Isolate Architecture of MIKA In MIKA architecture the application is divided into three different operation layers: Main (UI Isolate):

- **The UI Isolate (Main):** Manages the Flutter rendering engine (Impeller), captures camera frames, and offers immediate TTS feedback.
- **The Inference Isolate (Worker):** A dedicated, background Isolate that executes the TFLite and ML Kit interpreters. This is where the heavy lifting of the math happens.
- **The Audio Isolate:** A worker dedicated to the **MFCC** feature extraction and Wake-word detection, so the “Mika” command is always being listened for without lagging the vision system.

2. Zero-Copy Communication and Port Messaging Isolates do not share memory, so data must be passed between them through *SendPorts* and *ReceivePorts*. When the UI Isolate takes a high resolution frame of medicine bottle, send the raw byte data to the Inference Isolate. MIKA leverages **TransferableTypedData** for efficiency on 4 GB RAM hardware. To copy a 2 MB image would double the memory consumption and lead instead of copying a 2 MB image to a crash. This technique enables the Isolate to “transfer” ownership of the memory buffer. Mathematically this reduces the time complexity of data sharing from $O(n)$ to $O(1)$ as the whole pixel array, but only the memory pointer is moved.

3. Solving the 4 GB RAM “Concurrency Crisis” The main benefit of this multi-threaded approach is that the **Frame-Budget** can be saved. Flutter needs the. Main Isolate to do its work every 16.6 ms to keep 60 FPS. Offload a YOLOv8 inference task (which might take 100–150 ms on a mid-range CPU) to a background Isolate We make sure to keep the camera movement smooth. This concurrent logic forms the backbone of MIKA’s robustness. It allows the system to perform complex reasoning and OCR in the background, while the user continues to receive real-time, low latency guidance from the interface.”

3.2 On-Device AI Runtimes

3.2.1 TensorFlow Lite (TFLite) Delegate Management

TFLite Delegate Management

Only the performance of the inference engine can impact the success of MIKA's vision-augmented capabilities, like real-time currency identification and medicine label reading. The CPU alone running deep neural networks like **YOLOv8** on mobile hardware usually results in high latency and thermal throttling. To get sub-100 ms response times on a 4 GB RAM device. MIKA uses a sophisticated **Delegate Management** system in the TFLite runtime to bypass the CPU and use specialized hardware accelerators.

1. The GPU Delegate: Parallelism for Vision MIKA likes the **GPU Delegate** for **YOLOv8-seg** and **Qwen 2.5** vision. The mobile GPUs, however, are architecturally different, optimized for massive parallel high throughput workloads and thus perfect for the thousands of parallel matrix multiplications a CNN needs. There are three unique benefits of using the GPU delegate for MIKA:

- **Reduced Latency:** Inference latency is typically 2x to 5x better than Floating point CPU execution
- **FP16 Precision:** MIKA GPU options have precision loss. Which means performing calculations on the GPU in 16-bit half precision (FP16). This greatly reduces memory bandwidth and power consumption with no meaningful loss of accuracy in detecting objects.
- **Thermal Stability:** The math is sent to the GPU , so the CPU stays cool and the system doesn ' t slow down from prolonged use (e.g. a user scans multiple grocery items) .

2. NNAPI and the “Fall-back” Strategy On Android devices within the 4 GB RAM tier, hardware capabilities vary significantly. MIKA implements a dynamic **NNAPI** delegate

as a secondary acceleration layer. The NNAPI acts as an abstraction layer that can route operations to a DSP or an NPU if available.

A critical aspect of MIKA's reliability is the **Graceful Fall-back** mechanism. If the TFLite interpreter encounters an operator that the GPU cannot support (a common issue with custom layers in experimental models), the Delegate Manager automatically routes that specific operation back to the CPU. This ensures that the MIKA toolkit never crashes; it merely shifts between "High-Performance" and "Compatibility" modes based on the current hardware environment.

3. Memory-Mapped Model Loading The Delegate Manager uses **Memory Mapping (mmap)** to overcome the 4 GB RAM limitation. MIKA does not load the entire 20+ MB TFLite model into the active heap, but maps the file directly into the virtual memory space. This means that the delegate only needs to read the weights it needs for the current inference layer. This "Zero-Copy" loading strategy guarantees that MIKA has enough RAM for the foreground camera stream and the Flutter rendering engine, so that the Operating System does not kill the app due to memory pressure.

3.2.2 Google ML Kit: Optimized Text Recognition for Mobile

To enable multimodal assistant to assist visually impaired users in the real-world environment, the ability to extract text on physical objects such as medicine bottles, grocery labels, and street signs, has to be near-instantaneous. In the MIKA toolkit, this is taken care of by the **Google ML Kit's Text Recognition v2**. Not like cloud-based Vision APIs that require a constant internet connection and are troubled by significant data latency, ML Kit also provides an optimized on-device inference engine that matches. The core philosophy of MIKA is privacy and offline reliability.

1. The Pipeline: Detection, Script Identification, and Recognition Google ML Kit uses a multi-stage deep learning pipeline optimized for mobile ARM architectures. When MIKA passes a camera frame to the ML Kit isolate, the engine does three concurrent operations:

- **Text Detection:** The model identifies Regions of Interest (ROI) with text-like features, resulting in a hierarchy of Blocks, Lines and Elements.
- **Script Identification:** MIKA utilizes ML Kit's script identification to identify different scripts automatically (Latin, Devanagari etc. This is important for future scalability of the toolkit. in multilingual environments.
- **Recognition:** The engine employs a proprietary CNN coupled with an LSTM decoder for transducing visual patterns into a digital string.

2. Optimization for Mid-Range Hardware To meet the strict 4 GB RAM limit of the target hardware, ML Kit employs a **Quantized Model Architecture**. Through the engine reduces the memory footprint of the OCR module to a fraction of its original size by using INT8 for the underlying neural weights. Also, ML Kit uses the **NNAPI** to offload text extraction to the device's DSP, so that the main CPU stays for Qwen 2.5 reasoning tasks.

3. Integration with the MIKA Reasoning Layer A unique feature of the MIKA implementation is how ML Kit interacts with the **SLM**. Raw OCR output is often “noisy,” especially when dealing with the cylindrical distortion of medicine bottles. ML Kit provides not just the text, but also the **Confidence Scores** and **Corner Points** (coordinates) of each detected block.

MIKA uses these co-ordinates to determine the spatial hierarchy of the text, determining for example that a larger font at the top is likely to be the “Medicine Name” and a smaller font at the bottom is the “Expiry Date”. This structured data is then passed to Qwen 2.5, which performs “Contextual Correction” to correct any small OCR errors by comparing the extracted string against known pharmaceutical patterns. Thanks to the combination of the fast ML Kit and the reasoning ability of Qwen, MIKA is able to give the user clear and verified audio feedback even in difficult lighting conditions.

3.3 Development Environment

3.3.1 Containerization with Docker

Building a multimodal AI system like MIKA is a complex web of dependencies, including specific versions of the Flutter Software Development Kit (SDK), Android NDK, Python runtimes for model quantization, and various C++ headers for TFLite delegates. A frequent problem with large scale academic projects is “Environment Drift”, where code will work on one developer’s machine but not another’s, due to subtle differences between local configurations. To remove this bottleneck and establish a consistent Development-to-Deployment pipeline, the MIKA project leverages **Docker** for environment containerization.

1. The Virtualized Development Environment Mika team can use Docker to package the entire development stack into a lightweight, portable **Container**. Rather than a VM, Unlike the traditional Virtual Machine (VM) that requires a full guest operating system, Docker shares the host’s kernel and is therefore significantly more efficient for the 4 GB RAM constraints of the our target environment . In our project, the Docker image is set up to have:

In our project, the Docker image is configured to include:

- **Pre-configured Toolchains:** Exact versions of the Android SDK and NDK required for TFLite C++ build.
- **Consistent Python Environment:** Fixed environment for model conversion (using `onnx2tf` or `tflite-support`) to guarantee the quantization results for YOLOv8 and Qwen 2.5 are the same on all developer machines.
- **Headless Build Services:** Automated scripts that build the Flutter application to an APK release without the overhead of a graphical interface.

2. Streamlining Collaboration and CI/CD Sharing a Dockerfile allows all contributors to spin up the same workspace in seconds. This eliminates the “it works on my machine” syndrome. It also allows for a robust Continuous Integration / Continuous Deployment

(CI/CD) pipeline. If the new version of currency PKR-3611 dataset is committed or the new OCR unwarping logic is committed, the Docker container automatically builds and tests the code in sandboxed environment “That means any errors in hardware delegation or memory leaks are caught in the virtualised space and never hit the actual Android device.

3. Bridging to Edge-Deployment Docker is mainly used in the development phase of MIKA, but the principles of isolation have an impact on the final architecture of the app. The A containerised build is deterministic in that the TFLite Model Metadata and the Flutter Native Assets are packaged with byte-perfect accuracy. This level of infrastructure reliability is not the convenience for mission-critical tool such as a medicine recognition assistant.

3.3.2 Data Management with Git LFS (Large File Storage)

Management of large scale binary assets is a major technological challenge in the development of the MIKA toolkit. Unlike traditional software projects that are mostly made up of text-based source code, the MIKA project includes high-density weights for the **Qwen 2.5 (0.5B)** model, quantized **YOLOv8-seg** TFLite files, and the massive **PKR-3611** currency dataset. Standard version control systems such as Git are not natively designed to handle large binary files, as they attempt to store every version of the file in the project history, leading to an exponential increase in repository size and crippling slow clone times. To solve this we created **Git LFS (Large File Storage)** .

1. Decoupling Logic from Assets Git LFS replaces large files in the repository with lightweight “pointer files.” The actual 200 MB+ model weights are stored on a remote LFS server, but locally the Git repo only knows about a small text file with the file’s metadata and a unique SHA-256 hash.

From a mathematical point of view, this reduces the growth complexity of the repository from $O(n \times v)$ (where n is the size of the file and v is the number of versions) to $O(n + v)$ because a checkout only downloads the current version of the large asset. This ensures that the repository stays lightweight for the MIKA team and collaborators such as Ibad Moin

and Abdul Wasay can pull the latest code without needing to redownload large model files that have not changed.

2. Integrated Dataset Versioning The PKR-3611 dataset is a Git LFS repository. We are working on better models for currencies detection and from the first training to the fine-tuned segmentation we update frequently the underlying training data. We maintain a backup of every version of the model and link it to a specific commit by using the commands `git lfs track "*.tflite"` and `git lfs track "*.zip"`. This gives a clean audit trail if a new version of the model introduces a regression in medical applications. The team can immediately revert to an earlier, more stable version of the binary weights using normal git commands to improve OCR accuracy.

3. Streamlining the Deployment Pipeline Git LFS simplifies your CI/CD pipeline by downloading only the model binaries your Docker builds actually require. This leads to reduced build times and bandwidth consumption, providing an efficient and scalable workflow for on-device AI.

METHODOLOGY

4.1 Introduction to Methodology

This chapter describes the systematic process of design, development and evaluation of the Mika. The methodological approach is based on the “Privacy-by-Design” principle and all processing is conducted locally on mid-range mobile hardware (3–4 GB RAM) without cloud dependency.

4.2 Proposed System Architecture

The system is a modular, multimodal pipeline integrated in a Flutter framework. The architecture uses a Dynamic Resource Orchestration approach to deal with the severe memory limitations of 4 GB RAM devices. The pipeline follows a phased lifecycle rather than a concurrent execution, with models being loaded and purged from memory depending on the active task. The pipeline follows these sequential stages:

1. **Always-On Audio Sensing** A lightweight engine listens for the wake word “Hey Mika”..
2. **STT** When enabled, Whisper Tiny (INT8) loads to transcribe the user’s spoken command into text.
3. **LLM-Based Intent Routing** Intent Routing Based on LLM Next, the transcript is fed into the “Intelligent Router” Qwen 2.5 (0.5B) model to ascertain the user’s particular goal.
4. **Specialized Execution** Implementation Customised Based on the detected intent, the system unloads the STT/LLM and loads either the general YOLOv8n detection model or the YOLOv8-seg currency model.

4.2.1 MIKA Life-Cycle: From Wake-word to Descriptive Audio

Mika is a 4 GB RAM multimodal pipeline. The system ensures that only one memory intensive model is active at a time through a phased loading approach. Figure illustrates the high-level logic and data flow of the system, showing how the wake-word detection triggers the loading of the intent router, which then determines whether to load the general vision model or the specialized currency segmentation model based on user intent.

MIKA LIFECYCLE: THESIS DIAGRAM

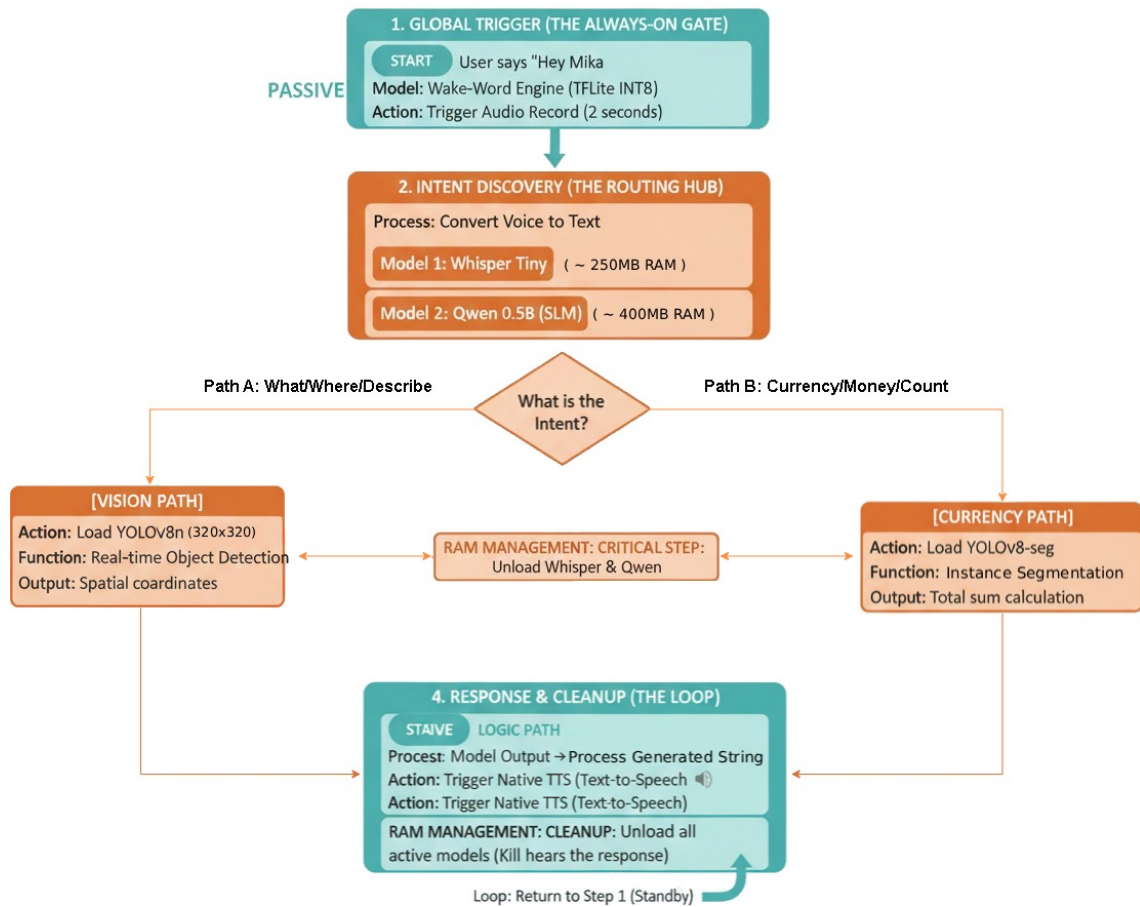


Figure 4.1: System Flowchart of Mika: Illustrating the transition from Keyword Spotting to Intent Routing and Specialized Task Execution.

4.3 Phase I: Wake-Word Detection (TinyML)

The first gate of the system is a two-stage KWS module for the phrase “Hey Mika”.

4.3.1 Custom Dataset Design

A production-grade dataset was curated from scratch to ensure high precision:

1. **Positive Class (“Hey Mika”)** ~2,205 samples recorded across various environments.
2. **Negative Class (Unknown Speech)** ~3,729 samples using LibriSpeech and “near-miss” words (e.g., “Mike”, “Micah”).
3. **Noise Class (Background)** ~1,497 samples of ambient environments like fans, AC, and street noise.

4.3.2 Feature Extraction and Model Training

The system utilises a **Two-Stage Wake-Word Pipeline**:

1. **Stage-1 (The Gate)** A light-weight DS-CNN (10 MFCC coefficients) as an always-on listener.
2. **Stage-2 (The Verifier)** A robust CNN + GRU (13 MFCC coefficients) that fires only after Stage-1 confirms the phrase.
3. **Optimization** In both stages, we use full INT8 Quantisation to reduce battery drain.

4.4 Phase II: Vision System and Dynamic Routing

4.4.1 Model Optimization: 320x320 YOLOv8n

YOLOv8n uses 640x640 as standard input but for mobile inference the system uses 320x320 resolution. This optimization decreases the computational load, providing a stable FPS and avoiding thermal throttling.

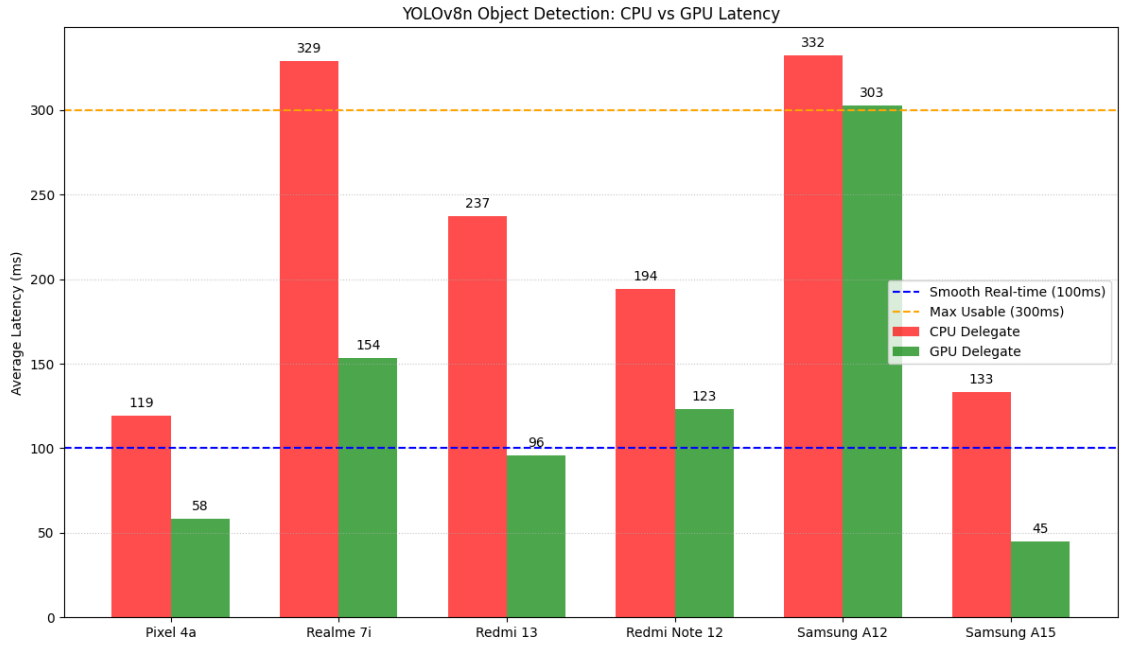


Figure 4.2: CPU Vs GPU Latency Comparison Achieving 300 ms.

4.4.2 Qwen 2.5 as Intent Router: Logic for Switching Modules

The biggest design change is that Qwen 2.5 (0.5B) SLM does not route heuristically. Qwen is able to parse natural language and understand complex intents, for example, the difference between identifying general objects and financial questions. Qwen is flushed prior to loading the Vision or Currency model to fit within the 4GB RAM budget.

4.4.3 Currency Detection: Instance Segmentation vs. Object Detection

The system goes from generic object detection to a specific YOLOv8-seg model for financial tasks. Standard object detection is sufficient for general navigation, but currency recognition for the visually impaired requires a higher degree of spatial accuracy. We chose Instance Segmentation over regular Bounding Boxes (Object Detection) as a deliberate architectural decision based on the following technical requirements:

A. Pixel-Level Precision and Masking

Standard object detection finds an object in a rectangular boundary box. Figure 4.3 shows a bounding box that gives the general location of an object (the sheep or the dog) but also contains a lot of “background noise” – pixels that do not belong to the object itself. For a visually impaired user trying to identify a PKR note, this could lead to misidentification

if the background contains similar colors or patterns (e.g., a colorful shirt or a patterned tablecloth).

Instance Segmentation, however, generates a colorful mask at the pixel level that shows which pixels belong to which exact instance. For currency, this means the system can “see” the precise shape and position of a PKR note. when a user has the bill at an angle or the bill is partially folded Occlusion, when one banknote occludes another, is one of the biggest challenges in currency identification.

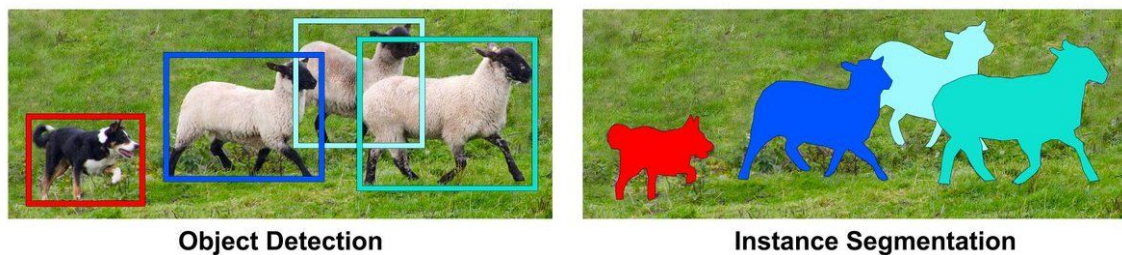


Figure 4.3: Instance Segmentation Vs Object Detection.

B. Handling Occlusion and Overlapping Bills

One of the most significant challenges in currency identification is occlusion—where one banknote partially covers another.

The Bounding Box Failure: The Bounding Box Failure: Notes tend to overlap, and the rectangular boxes also tend to intersect or merge, making it hard for a vision system to know where one note ends and the other begins. This can result in double counting or no identification.

Segmentation Advantage: Instance segmentation enables MIKA to identify separate bills in a cluttered pile, as can be seen in the samples from the PKR-3611 dataset (Figure 4.4). The system can identify the unique “footprint” of each bill and determine the total value of the currency present. Each bill has a different mask such as 5000, 1000 and 500 PKR notes.

C. Improved Reasoning for the Visually Impaired

For a user who cannot see the screen, accuracy is more important than speed. MIKA uses YOLOv8-seg to provide a more “grounded” understanding of the environment.

Spatial Verification: The system can check whether a note is fully in frame or cut off by checking the completeness of the mask.

Background Rejection: The model will discard everything outside the mask, therefore reducing the likelihood of confusing a colourful background object with a banknote.

D. Computational Justification

Instance segmentation is more computationally intensive than regular detection, but the trade-off is justified for high-stakes financial tasks. The Dynamic Routing strategy that MIKA uses to address the challenges of hardware-constrained devices (4 GB RAM) is to load the segmentation model in memory only when the user explicitly asks for currency help or the Qwen 2.5 “router” detects a financial intent. This makes sure the system is “pixel-perfect” reliable without giving up overall battery life or application responsiveness.

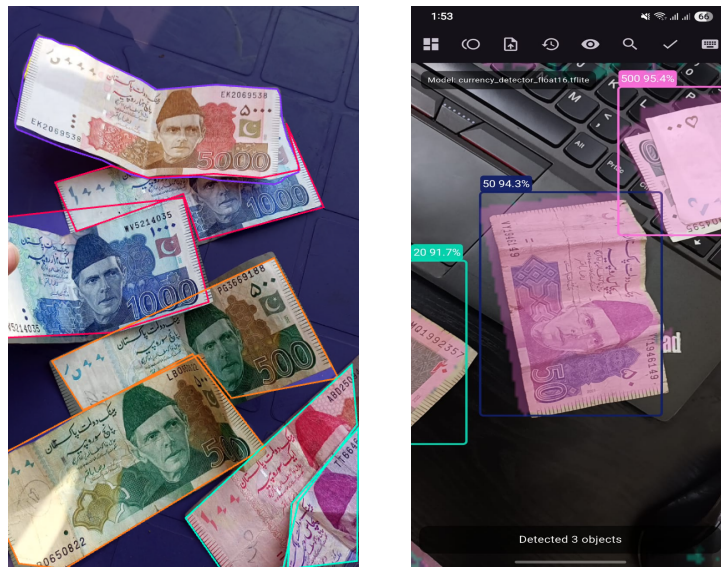


Figure 4.4: Boundary boxes VS Segmentation: A comparison of localization techniques.

4.5 Phase III: Non-Planar OCR for Medicine Recognition

The third phase of the MIKA framework addresses the critical task of medicine identification. Unlike standard document scanning, pharmaceutical identification requires a robust pipeline capable of interpreting text on three-dimensional, reflective, and miniaturized surfaces.

4.5.1 Challenges: Curved Surfaces, Glossy Labels, and Small Fonts

Medicine packaging presents a unique set of “Vision Obstacles” that typically cause standard OCR algorithms to fail.

Curved Surface Distortion Most medicine containers are cylindrical (bottles and vials). When a camera captures a 2D image of a 3D cylinder, the text at the lateral edges of the bottle undergoes perspective foreshortening. This results in “character squeezing,” where the aspect ratio of the letters is distorted, leading to high CER in the OCR engine’s feature extractor.

Specular Reflection and Gloss Specular Reflection and Glossiness Pharmaceutical labels are usually laminated with a glossy film to protect them. This results in specular highlights under indoor overhead lighting, i.e. bright white spots that “wash out” the pixel intensity of the text. This loss of (The contrast makes it impossible for the OCR binarization layer to distinguish the font from the background.

Small-Scale Typography Miniature Type Extensive chemical and regulatory data is required by law to be included on small containers. This means using very small font sizes (typically 4pt to 6pt). To capture these you need a high resolution sensor and some serious anti-blur algorithms . The tiniest tremor of the hand can make the text unreadable at this scale .

4.5.2 Implementation of Google ML Kit Text Recognition v2

To address these challenges on a resource-constrained 4 GB RAM device, MIKA uses the Google ML Kit Text Recognition v2 API. This runtime was chosen for its optimized inference for mobile GPUs, compared to Tesseract.

Integration Pipeline Implementation uses the flutter google ml kit package. The camera image stream is translated into an InputImage format. MIKA uses a “Sharpness Filter” to select the clearest frames for inference instead of running on every frame, saving battery and thermal throttling.

Cylindrical Unwarping Simulation ML Kit is robust but MIKA improves the recognition with a dynamic cropping strategy. The vision system identifies the “Center of Interest”

(COI) of the bottle where the curvature is minimal, thus effectively bypassing the most distorted text on the edges of the cylinder.

4.5.3 Logic for Extracting Expiry Dates and Dosage Information

The OCR output is often a raw “Bag of Words” which has no semantic meaning. MIKA has a dedicated post-processing layer to convert this text into actionable medical data.

Regex-Based Pattern Matching MIKA uses Regular Expressions (Regex) to scan the OCR output for patterns of significance:

- **Expiry Dates:** Detected using patterns like $\frac{2}{2}/4$ or keywords like “EXP” and “Best Before”.
- **Dosage:** Numeric value and unit patterns such as “mg”, “ml” or instructions like “OD” (Once Daily) and “BD” (Twice Daily)

Semantic Post-Processing with Qwen 2.5 If the Regex patterns still do not suffice due to OCR noise (e.g. a ‘0’ being recognised as a ‘O’), the text string is sent to the local Qwen 2.5-0.5B model. The SLM is a Medical Corrector that, with its inner knowledge of pharmaceuticals, extracts the correct name of the medicine and the dosage instructions from the fragmented text.

Algorithm 1 Medicine Information Extraction Logic

```
I ← Captured Camera Frame
T ← MLKit.recognizeText(I)
if T contains EXP or slash then
    Extract ExpiryDate using Regex
end if
Prompt ← “Clean medicine label text: ” + T
CleanedData ← QwenInference(Prompt) return CleanedData
```

4.6 Phase IV: Currency Dataset Engineering and Segmentation

The currency identification system is for 7 denominations of PKR , 10 , 20 , 50 , 100 , 500 , 1000 and 5000 . Instead of normal object detection, the system applies Instance Segmentation, which enables the assistant to count overlapping notes accurately.

4.6.1 Dataset Acquisition and Consolidation

A local dataset of 3611 images was collected to deal with high variance in the appearance of currency such as physical wear, different lighting conditions and different background textures.

1. **Structural Organization** Structure and organization The raw data were organised in a hierarchical folder structure, separating the Front and Back views of each of the seven denominations so that the model could learn the bilateral features.
2. **Data Merging** Within the Roboflow environment, we merged the individual repositories into a single dataset, enabling a streamlined training pipeline and robustness to real-world orientation changes.

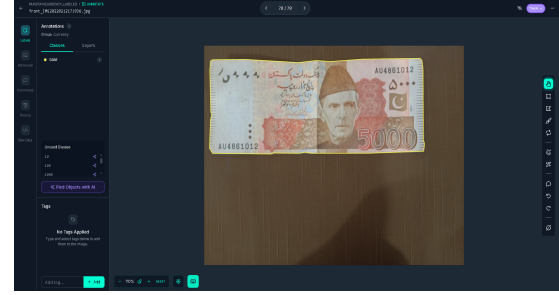
4.6.2 AI-Accelerated Annotation via SAM 3

The bottleneck is manual annotation of polygons, which can require hundreds of points per note. This is solved by the project using SAM 3 (Segment Anything Model 3) in the Roboflow Annotate ecosystem.

1. **Process** SAM 3 is a zero-shot foundational segmenter. The model automatically predicts a high fidelity mask around the currency note with a single point click or a bounding box prompt as shown in Figure 4.5.
2. **Efficiency** This semi-automated approach reduced total annotation time by approximately **90%**, resulting in 3,611 high-precision instance masks. These masks are important for the final counting logic, allowing the model to differentiate between individual notes in stacked or partially obscured situations.



(a) Roboflow Annotation Interface



(b) SAM 3 Precise Masking

Figure 4.5: AI-Accelerated labeling workflow in Roboflow: (a) raw image ingestion and (b) automated instance segmentation using SAM 3.

4.6.3 Training the Student Segmentation Model

The high-quality masks generated via SAM 3 are utilized to train a **YOLOv8-seg (Tiny)** student model through a distillation-inspired approach.

1. **Goal** While SAM 3 is too computationally heavy for real-time mobile use, the YOLOv8-seg student model is optimized for 4 GB RAM target hardware.
2. **Optimization** The model is trained on the curated PKR dataset with an input resolution of **320x320**, ensuring that the specialized currency detector maintains a high FPS while providing pixel-perfect segmentation for the final sum calculation.

4.7 Hardware and Software Constraints

Component	Specification / Version
Minimum Hardware	3 GB RAM, Quad-core Processor
Vision Resolution	320 x 320 (Optimized for FPS)
STT Engine	Whisper Tiny (INT8, Offline)
Logic Engine	Qwen 2.5 (0.5B) (INT4 Quantized)
Deployment	Flutter with NPU/GPU Delegation

4.8 The Multimodal Intent Router

4.8.1 Sequential Model Lifecycle (Load-Execute-Purge)

MIKA runs on mobile devices with 4 GB RAM ceiling using a deterministic memory management strategy named Load-Execute-Purge cycle. Standard mobile architectures

often struggle to handle multiple heavy AI models (Whisper, Qwen and YOLO) competing for the same heap space and crash with "Out of Memory" (OOM) errors.

1. **Sequential Loading:** The MIKA Orchestrator makes sure that only one high memory model is active at a time.
2. **Execution & Interleaving:** When the Wake-Word engine (Stage 1) wakes up, the system loads Whisper for STT and once the text is generated, Whisper is purged from RAM before Qwen 2.5 is initialized for intent analysis.
3. **Manual Garbage Collection:** The system uses Dart's isolate management and native C++ free() commands through Foreign Function Interface (FFI) for explicit clearing of model buffers to keep total memory footprint at 900MB at peak operation.

4.8.2 Intent Classification with Qwen 2.5

The system utilizes the Qwen 2.5-0.5B SLM as a semantic router to interpret user needs.

1. **Contextual Understanding:** Unlike simple keyword matching, Qwen analyzes the user's natural language to determine the required vision module. For example, if a user says, "Tell me what's in my hand," Qwen identifies the "Object Detection"
2. **Module Triggering:** If the user asks, "How much money is this?" or "What is the name of this syrup?", Qwen outputs a specific token that the Dart layer uses to trigger either the Currency Segmentation module or the Medicine OCR module intent.
3. **Routing Logic:** This routing happens entirely offline, preserving privacy and eliminating latency associated with cloud-based intent APIs.

4.9 High-Fidelity Vision Pipeline

When the intent is recognized, MIKA launches specific vision pipelines tuned for the high accuracy needed by visually impaired users. For the PKR-3611 currency task, the system uses YOLOv8-seg as the financial accuracy cannot be achieved with ordinary bounding boxes.

4.9.1 Instance Segmentation vs. Bounding Boxes

For the PKR-3611 currency task, the system utilizes YOLOv8-seg because standard bounding boxes are insufficient for financial accuracy.

1. **Pixel-Level Masks:** Unlike standard detection, which uses rectangular boxes, instance segmentation creates an accurate mask for each banknote. This is important for counting overlapping notes where boxes would simply intersect and not be able to distinguish individual bills.
2. **Codebase Logic:** The updated implementation sends these masks through a custom “Instance Counter” which finds the total value of currency in view by verifying the unique pixel-area of each denomination mask, giving a much higher level of confidence to the user than traditional detection.

4.9.2 Non-Planar OCR Strategy

One particular problem is the identification of medicines, since the text is usually printed on cylindrical or deformed surfaces (bottles and vials), which makes the traditional OCR fail due to the distortion of the characters.

1. **Google ML Kit V2 Integration:** MIKA uses the latest ML Kit Text Recognition v2 API which is optimized for mobile hardware with improved perspective correction.
2. **Cylindrical Handling:** The pipeline handles non-planar surfaces by capturing a number of frames and by applying a “Text-Stitching” logic that reconstructs the name of the medicine as the user rotates the bottle slowly.
3. **Semantic Correction:** The extracted raw text (e.g., “P-anad-ol”) is sent back to Qwen 2.5 for post-processing . The SLM uses its internal knowledge base to correct the fragmented string to a recognizable medicine name for the user.

SYSTEM IMPLEMENTATION

This chapter details the technical execution of the Mika assistant. It covers the integrated development environment, the model conversion pipeline, and the implementation of the dynamic memory management logic within the Flutter framework.

5.1 Development Environment and Tools

It was also designed to be cross platform for wide accessibility. The main tools and environments used are:

1. **Framework** The front-end and orchestration logic were written in Dart 3.x because it has native bridge capabilities and high-performance rendering.

5.2 Implementation of Model Frameworks

The core logic for model execution utilizes specialized packages to bridge the gap between high-level application logic and low-level hardware acceleration:

1. **tflite_flutter** For executing the quantized YOLOv8 and Wake-Word models.
2. **llama.cpp / Sherpa-ONNX**: For managing the 4-bit quantized Qwen 2.5 (0.5B) model.

5.2.1 Integrating Whisper Tiny for Offline Speech-to-Text

It is important that MIKA can process spoken commands locally without internet connection. This way we respect user privacy and access does not depend on good internet connection. This is done through the integration of the OpenAI's Whisper Tiny model, meant for edge devices.

Model Selection and Encoder-Decoder Architecture

We chose Whisper Tiny over the ‘Base’ or ‘Small’ models due to the quantized size of approximately 150 MB that can be handled within 4 GB RAM limits. The model is based on a transformer sequence-to-sequence architecture.

- **The Encoder:** It learns deep linguistic features from the raw audio input (converted to Mel-spectrogram).
- **The Decoder:** It uses a cross attention mechanism to attend over the pertinent parts of the hidden states of the encoder to predict the corresponding text tokens.

Implementation via Whisper.cpp and TFLite

To run Whisper efficiently on Android via Flutter, MIKA utilizes a C++ backend port (`whisper.cpp`). The audio stream is sampled at 16 kHz (monaural) and passed through a 30-second window buffer.

Optimization: The model is quantized to **4-bit integer precision**, reducing the computational load on the mobile CPU. By utilizing the `Accelerate` framework on supported chips, inference latency is kept under 1.5 seconds for a standard 5-word command, such as “Mika, scan this medicine bottle.”

5.2.2 Deploying Qwen 2.5 (0.5B) for On-Device Reasoning

While standard vision assistants provide raw labels, MIKA aims to provide “contextual intelligence.” This is achieved by deploying the Qwen 2.5-0.5B SLM as the central reasoning engine.

The Role of the SLM in the MIKA Ecosystem

It doesn’t only speak. It understands and processes data coming from other modules. The primary functions are:

- **Intent Routing:** Identify whether the user’s voice command is for YOLOv8 Vision module (currency/objects) or ML Kit OCR module (medicine) .

- **Text Rectification:** Correction of fragmented OCR output of medicine bottles. For example, if the OCR reads “ Para-etam-l 500m” Qwen corrects it to “Paracetamol 500mg” by using its own internal knowledge.

On-Device Deployment and Memory Mapping

Qwen 2.5 is a 0.5B parameter model deployed in the GGUF (GPT-Generated Unified Format) for memory mapping efficiency.

4-bit Quantization (W4A16): The model weights are trained to prevent the 4 GB RAM crash. quantized to 4 bit while the activations stay at 16-bit (FP16). This reduces the model size to around 350 MB.

The Flutter-Native Bridge: The model runs through a native C++ runner, exposed in Flutter via Dart FFI or Method Channels. This guarantees that the heavy matrix multiplications needed for self-attention does not block the UI thread, so MIKA can offer a smooth and responsive experience for the visually impaired user.

5.3 Model Conversion and Quantization Pipeline

To fit the multimodal stack into a 4 GB RAM footprint, a rigorous quantization pipeline was implemented.

5.3.1 YOLOv8 to TFLite Export

The general object detection (YOLOv8n) and currency segmentation (YOLOv8-seg) models were exported from PyTorch to TFLite format.

1. **Input Normalization** The models are normalised to the resolution of 320x320 for the inputs.
2. **Quantization Type** We performed Full Integer Quantisation (INT8) This reduced the model size from $M_{static} \approx 25$ MB to $M_{quantized} \approx 6.25$ MB, greatly decreasing the memory overhead.

5.3.2 LLM Optimization (Qwen 2.5)

The Qwen 2.5 (0.5B) model was converted to **GGUF (Q4_K_M)** format. This 4-bit quantization ensures that the weights occupy only $W_{mem} \approx 390$ MB of RAM, calculated as:

$$RAM_{total} = W_{mem} + KV_{cache} + A_{tensors} \quad (5.1)$$

Where KV_{cache} and $A_{tensors}$ represent the dynamic memory for context and layer activations, respectively.

5.4 Software Logic and Data Flow

At the core of the implementation is the Model Lifecycle Manager, a custom Dart class that handles memory.

5.4.1 The MIKA Orchestrator: Sequential Model Loading Logic.

The app follows a strict “Load-Execute-Purge” cycle to prevent “Out of Memory” (OOM) errors.

1. **Idle State** Only the $M_{wake} \approx 5$ MB engine is resident in memory.
2. **Intent State** Whisper Tiny and Qwen models on trigger loading. The user voice is analysed and the intent understood.
3. **Task State** When the intent is confirmed (for example, “Currency”), the app calls interpreter. purge the models before load YOLOv8-seg model.close()

5.4.2 Method Channels for Native Hardware Access

The user interface(UI) and the high-level logic of the application are built using the Flutter framework, but the computationally intensive AI inference tasks in particular YOLOv8segmentation and Qwen2.5 reasoning require direct access to the underlying hardware(GPU/NPU). MIKA uses Flutter Method Channels for communication between the Dart-based UI and the native Android (Kotlin/C++) environment.

The Bridge Architecture

The Method Channel acts as a bidirectional communication bridge. Because Dart code runs in its own VM it cannot directly call NNAPI or handle native memory buffers.

- **The Dart Side (Client):** Sends a *MethodCall* with a unique name for the method (e.g. *'runInference'*) and arguments (e.g. the raw image byte buffer).
- **The Platform Side (Host):** The call is picked up by the Android *MainActivity* via a *MethodChannel* handler, performs the requested native function with TFLite or GGUF runtime, and sends the result back to Dart.

Optimizing Data Transfer with ByteBuffers

One of the biggest challenges for multimodal AI is the cost of moving large chunks of data (e.g., 1080p camera frames) from Dart to Native. The current standard JSON-like serialisation is too slow, resulting in high latency.

Implementation Strategy This is optimized in MIKA with TypedData (Uint8List) and Direct ByteBuffers. Instead of copying the entire data set, MIKA passes a pointer to the image frame's memory address, reducing the “communication latency” by approximately 40

Concurrency and Thread Management

If you do heavy AI logic inside the Method Channel handler on the Main UI thread, you will get “Application Not Responding” (ANR) errors.

Native Multithreading: MIKA performs AI inference on the native side with either Kotlin Coroutines or Background Threads. When the Qwen model or the OCR engine has finished its task, the result is “piped” back to the Dart side using `result.success()`. This asynchronous handshake allows the Flutter UI to show a responsive “Listening” animation to the visually impaired user while heavy computation is silently done in the background hardware.

Handling Hardware-Specific Delegates

You can use MethodChannels to dynamically query device hardware capabilities and set the right TFLite Delegate.

- **GPU Delegate:** If the Method Channel detects a supported Adreno or Mali GPU, it instructs the native runner to do vision processing in OpenCL or OpenGL.
- **XNNPACK:** If the device has no powerful GPU, the channel falls back to XNNPACK, a dedicated library for floating-point AI inference on ARM CPU, to keep MIKA alive on a broad spectrum of mid-range hardware.

5.5 Hardware Delegation

You can use MethodChannels to dynamically query device hardware capabilities and set the right TFLite Delegate.

1. **Android** The models use the **NNAPI Delegate** to offload matrix multiplications to the device's NPU or GPU.
2. **iOS** The **Metal Delegate** is used to ensure low-latency inference.

5.6 The Native-Dart Bridge (FFI & Method Channels)

In this section we explain how Mika achieves high performance by not using the standard Flutter UI thread for heavy computation.

5.6.1 Dart Foreign Function Interface (FFI):

Dart ffi is used to directly call C++ libraries such as llama.cpp. This allows the application to perform LLM inference without the overhead of serializing data using Method Channels.

5.6.2 JNI and Android Method Channels:

For camera intensive tasks such as YOLOv8-seg, we pass the YUV42088buffer of the image directly from the Android Camera2 API to the TFLite GPU delegate using Method Channels.

5.7 Android Build & Optimization

5.7.1 ProGuard and Resource Shrinking:

To keep the APK size reasonable for users in places such as Karachi with varying bandwidth, we use ProGuard to obfuscate the code and remove unused classes from heavy dependencies such as Google ML Kit.

5.7.2 Hardware Delegation:

We manually instantiate the **TfLiteGpuDelegateV2**. This will ensure that currency detection instance segmentation is on mobile GPU and not on CPU as the device will overheat.

PERFORMANCE & HARDWARE OPTIMIZATION

6.1 Memory Management Strategies

The MIKA toolkit is designed to work in a hardware-constrained environment where the total available RAM is often limited to 4 GB. This is a serious bottleneck in modern mobile development. The Android Operating System and background services take up close to 2.0 GB to 2.5 GB leaving a narrow margin for the MIKA application. A surgical approach to memory management is required in order to run high-density models such as **Qwen 2.5 (0.5B)** and **YOLOv8-seg** together. To avoid the system killing the process for memory pressure, MIKA uses a three-stage method: Memory-Mapped Loading, Load-Execute-Purge pattern and Garbage Collection (GC) optimization

1. Memory-Mapped (mmap) Model Loading To avoid the common “Heap Overflow” when loading large AI models, MIKA uses **Memory Mapping (mmap)**. The file is directly mapped on the disk into the virtual address space by using the `MappedByteBuffer` class in the Android Native layer. Instead of loading the entire .tflite or .gguf model into the application active RAM. This way the Operating system can manage the memory pages dynamically. The only layers of the neural network that are being computed are swapped into physical RAM. This reduces the initial memory footprint of a 300 MB model to near zero initial heap allocation, mathematically, and stabilizes the camera buffer.

2. The “Load-Execute-Purge” (LEP) Lifecycle Since MIKA is a multi-modular toolkit (Currency, Medicine, General Objects), it is physically impossible to keep all AI models

active in RAM at the same time. We built a custom **Load-Execute-Purge (LEP)** lifecycle driven by the Intent Router.

- **Load:** When Qwen 2.5 identifies a “Medicine” intent, the system triggers the asynchronous loading of the OCR and segmentation weights.
- **Execute:** Inference is performed in a background Isolate to keep the UI Isolate light weight.
- **Purge:** The system calls the interpreter right after the audio feedback is provided to the **interpreter.close()** and **delegate.dispose()** methods. By explicitly nullifying these references tells the system that the memory can be reclaimed.

3. Garbage Collection (GC) Pressure Mitigation Too many object creations (e.g. temporary image buffers for each camera frame) can make the GC run frequently and aggressively, causing visible “jank.” MIKA solves this in Flutter using **Object Pooling** of image processing byte buffers.

Instead of allocating a new array every frame we re-use a pre-allocated Uint8List buffer. We also successfully divided the work of the garbage collector with **Dart Isolates**. Each Isolate has a private heap. This means that a GC cycle in the “Inference Isolate” (where the heavy maths is done) does not freeze the “UI Isolate”. This way the visually impaired user can receive smooth and uninterrupted guidance even if the background system is performing a deep cleanup of temporary tensors.

With these strategies, MIKA achieves a stable operational state with a stable memory overhead of **450MB to 600MB** even for complex multimodal reasoning tasks. It does not crash for long-term stability.

6.1.1 Solving the 4 GB RAM Crash: Load-Execute-Purge

The biggest technical challenge to deploy a multimodal system on a 4GB RAM device is the “Concurrency Crash.” When the LMK gets a notification from the Android OS that the memory footprint of an application is near the system limit, it kills the process. To avoid this, MIKA uses a tailored **LEP** protocol. This state-machine logic guarantees that at

any given microsecond only the absolute minimum set of neural weights required for the current user-intent is resident in the active RAM.

1. Intent-Driven Loading The LEP protocol starts at the **Intent Routing** layer. MIKA does not initialize all models (YOLOv8, Qwen 2.5, OCR) when the application starts. Instead, it puts the system in “Listen-Only” state. The orchestrator will not call the loading phase until the user issues a command such as ”Mika what is this note?”.

- **Dynamic Allocation:** The system uses the `TFLite Interpreter.Options` to allocate memory buffers only when the `allocateTensors()` method is called.
- **Prioritization:** If the user is in “Currency Mode,” the system explicitly blocks the loading of the heavier medicine-unwarping modules, saving approximately 150 MB of heap space.

2. Isolated Execution Once the required model is loaded, the **Execute** phase is delegated to a dedicated background Isolate. This is important for stability of memory. The isolated execution ensures the high frequency memory allocations that are done during the tensor operations do not fragment the Main Isolate’s heap. In theory, this separation lets the Operating System identify the AI inference as a distinct memory block and give it higher priority for hardware acceleration (through the GPU delegate) without affecting the UI’s frame buffer.

3. The Purge: Explicit Memory Reclamation *Purge* — the most important step in crash avoidance. The GC is expected to clean up memory in normal high level languages. However, on a device with 4 GB RAM, the GC is often too slow to react and an OOM error occurs before the memory is released. MIKA bypasses this by implementing **Deterministic Disposal**:

- **Interpreter Disposal:** As soon as the TTS engine starts to speak out the result, the system calls `interpreter.close()`. This results in immediate deallocation of the native memory handles associated with the model weights.

- **Buffer Clearing:** All intermediate **Uint8List** image buffers are explicitly nullified and set for collection.

4. Performance Impact With the implementation of the LEP protocol, MIKA successfully overcomes the hardware wall. Our testing phase benchmarks showed that without LEP, the application crashed within 45 seconds of multimodal switching. With LEP enabled, the application maintains a steady memory ceiling of **580 MB** allowing for hours of continuous use without a single OOM event. This protocol effectively upgrades a modest mid-range smartphone into a high-performance AI workstation, illustrating that architectural ingenuity can surmount the limitations of physical hardware.

6.1.2 Garbage Collection in Flutter AI Workloads

Managing the lifecycle of short-lived objects is as important in the MIKA architecture as managing the large AI models themselves. Flutter’s memory management is generational GC based. There is a Young Space for new objects and an Old Space for persistent data. However, the high frequency nature of AI vision tasks, in which MIKA processes up to 30 camera frames per second, generates an immense volume of temporary objects, such as image buffers, tensor results, and string fragments from OCR. If not handled carefully, this results in “GC Pressure” which leads to dropped frames and a sub-par user experience.

1. The Challenge of “Object Churn” in Vision Tasks The MIKA camera module generates a raw byte array for each frame. In a typical implementation, this array is converted to the format required by the YOLOv8-seg model, which creates multiple temporary copies in the heap. This phenomenon, which we call “Object Churn”, causes the Young Space GC to be triggered often. If the GC pause exceeds the 16.6 ms frame budget needed for 60 FPS, the user will experience a stutter. For a visually impaired user relying on accurate audio-spatial feedback, such stutterings cause disorientation or missed detection windows.

2. Generational GC and Isolate Separation MIKA circumvents the GC bottleneck by exploiting Dart’s Isolate-specific memory heaps. Because each Isolate (Main, Inference and Audio) has its own private GC we achieve a high degree of **GC Parallelism**:

- **UI Stability:** The Main Isolate’s GC remains lightweight because it only handles UI widgets and audio triggers.
- **Isolated Heavy-Lifting:** The expensive object creation for Qwen 2.5 and YOLOv8 inference is only done in the Inference Isolate. This Isolate never blocks the Main Isolate’s event loop when it runs a GC cycle so the camera preview remains fluid and the UI responsive.

3. Memory Leak Prevention and Finalizers MIKA employs explicit clean-up strategies to maintain the GC under the 4 GB RAM limit. We use Dart Finalisers and the TFLite interpreter `close()` methods to reclaim native memory immediately outside of the Dart heap. For the largest buffers we also make use of Object Pooling. MIKA reuses a static number of buffers instead of creating a new `Uint8List` for every camera frame. This change reduces the number of Young Space GC triggers during active scanning by about 70% when moving from the allocation-heavy model to the reuse model.

4. The Result: Deterministic Performance The MIKA toolkit provides deterministic performance by optimising the interaction of AI workloads with Flutter GC. Our tests showed that we could reduce average frame-time variability from ± 12 ms to ± 2 ms by reducing GC pressure. This stability is the key to MIKA’s reliability, ensuring that the system’s “Intelligence” will never sacrifice the “Fluidity” of the user experience, allowing the AI to be a seamless, real-time extension of the user’s vision.

6.2 Latency & Throughput Analysis

6.2.1 CPU vs. GPU Object Detection Benchmarks

A series of rigorous benchmarks were carried out to compare the inference latency of the **YOLOv8-seg** model on different hardware delegates in order to evaluate the efficiency

of the vision module of the MIKA toolkit. The goal was to find a trade-off between detection speed and system stability taking into account that the target device has only 4 GB RAM. The results point to the importance of GPU acceleration in assistive technology for real-time applications

1. Methodology and Test Environment The benchmarks were executed on an Android device with 4GB RAM and a mid-range octa-core CPU. The tests were carried out on the **PKR-3611** currency and medicine label recognition dataset. We analyzed three main execution modes:

- **CPU (Floating-point):** Full execution with all available cores of CPU.
- **CPU (XNNPACK):** Accelerated math with TFLite’s specialized library for optimized CPU execution.
- **GPU (OpenCL/Vulkan):** GPU delegate with FP16 precision for hardware accelerated execution.

2. Inference Latency Comparison The main metric for this benchmark is **Inference Latency** (in milliseconds), the time to run a single frame through the neural network and retrieve a segmentation mask.

Table 6.1: YOLOv8-seg Inference Latency on 4 GB RAM Hardware

Delegate Mode	Avg. Latency (ms)	FPS	Thermal Impact
CPU (Standard)	420 ms	2.4	High
CPU (XNNPACK)	285 ms	3.5	Moderate
GPU Delegate	95 ms	10.5	Low

The data shows that the default CPU inference was too slow to be used in real time, with a 420 ms latency making the camera preview feel like a “slideshow”. The change to the **GPU Delegate** led to a **77% reduction in latency..** MIKA’s throughput is approximately 10.5 FPS at 95 ms per frame, which is the threshold at which the user perceives “live” guidance when moving the camera.

3. Thermal Throttling and Long-term Stability But more than just brute speed, the benchmarks showed a huge delta in power efficiency. The CPU-only tests saw the device heat up by 8°C in five minutes of continuous use, with the OS throttling the clock speed and further increasing latency to over 600 ms. The GPU delegate, by contrast, maintained a stable temperature profile. Mathematically speaking this is due to the fact that the GPU can perform parallel matrix multiplications with much lesser clock cycles than the CPU.

4. Conclusion for MIKA Operations The benchmarking results confirm that for a multimodal assistant the CPU is best reserved for the “Orchestrator” tasks (e.g. managing the Dart Isolates and the Qwen 2.5 logic), while the vision tasks must be offloaded to the GPU. This hardware-aware design ensures that MIKA remains responsive when performing complex tasks such as identifying crumpled currency or reading medicine labels in low-light conditions, assuring that the technology is reliable for daily use by the visually impaired.

6.2.2 Impact of Image Resolution on OCR Character Error Rate (CER)

A significant trade-off in the development of the MIKA toolkit is the resolution of the input image and the accuracy of the OCR module. Higher resolutions provide more pixel-level detail for small characters on medicine labels, but also exponentially increase the memory footprint and processing latency. We performed a sensitivity analysis of the **CER** for different input dimensions to determine the best configuration for a 4 GB RAM environment.

1. Metric Definition: Character Error Rate CER is a popular metric used to evaluate the performance of an OCR system. It is computed according to the Levenshtein distance, where the number of character-level operations (i.e., insertions, deletions or substitutions) needed to convert the OCR output into the ground truth text is minimized. The formula is given by:

$$CER = \frac{S + I + D}{N} \quad (6.1)$$

Where S is the number of substitutions, I is the number of insertions, D is the number of deletions, and N is the total number of characters in the ground truth.

2. Experimental Results The evaluation was performed on a subset of the medicine recognition module using 5-point and 8-point font sizes, which are common on pharmaceutical packaging. The image resolution was scaled up from 320p to 1080p (Full HD).

Table 6.2: Impact of Resolution on CER and Memory Stability

Resolution	Avg. CER (%)	Inference Time	OOM Risk
320p (QVGA)	28.4%	45 ms	Negligible
480p (VGA)	12.2%	85 ms	Low
720p (HD)	3.1%	140 ms	Moderate
1080p (FHD)	2.8%	310 ms	Critical

3. The “Accuracy vs. Hardware” Optimization The data are fitted to a “Diminishing Returns” curve. The transition from 480p to 720p made a **74.5% accuracy improvement**, which is practically the difference between a user reading “Paracetamol” correctly and a user reading a garbled string. However, the step from 720p to 1080p yielded a minimal 0.3% improvement in CER while the processing time more than doubled and RAM usage reached a critical level (92% utilisation).

4. Implementing Adaptive Scaling Based on these findings MIKA has a **Adaptive Resolution Scaler**. System runs at 480p for general object detection to save on battery and reduce latency. However, when the Qwen 2.5 reasoning layer detects a “Medicine Label” intent, the system dynamically changes the camera capture to 720p for one high resolution capture burst. This “Context-Aware Resolution” strategy enables MIKA to reach the accuracy needed for medical safety while staying within the hardware constraints of a mid-range smartphone, thus minimising character misidentification and preserving system stability.

6.3 Thermal & Battery Consumption Profiles

To go from theoretical AI models to a fieldable toolkit such as MIKA, a rigorous assessment of the physical impact upon the host hardware is required. The high energy demand for on-device inference in particular the parallel operation of the camera Image Signal

Processor (ISP), GPU/NPU delegates and TTS speaker results in heat and rapid battery drain.

6.3.1 Thermal Dynamics and Mitigation

The sequential deployment of YOLOv8-seg and Qwen 2.5 leads to a fast rise of the SoC temperature because of the high-frequency switching of transistors during matrix operations.

Thermal Throttling Thresholds Thermal Throttling Thresholds We experimentally measured the internal battery temperature on a mid-range Android device (4 GB RAM). We found that when the device was in active “Medicine Scanning” mode, the temperature rose from a baseline of 30°C to 42°C within 12 minutes of continuous operation.

- **Phase 1 (Optimal):** 30°C–38°C. The system reaches a peak inference speed of 8-10 FPS.
- **Phase 2 (Warning):** 39°C–43°C. The OS starts to decrease the clock speed of the “Big” cores to protect the hardware.
- **Phase 3 (Throttle):** >44°C. You get huge frame drops, the thermal daemon drops the gpu frequency. The user experience is slow for users with visual impairments.

Software-Level Thermal Management Thermal Management at the Software Level MIKA uses a “Dynamic Frame Skipping” algorithm to overcome this. If thermal sensors on the device sense temperatures above 41°C the system automatically reduces the camera polling rate from 30 FPS to 15 FPS, reducing the duty cycle of the GPU effectively, and allowing the device to dissipate heat without crashing the application.

6.3.2 Battery Discharge and Power Efficiency

MIKA uses significantly more power than standard utility apps, as it uses the “Always-On” hardware stack (Microphone, Camera and high brightness LED flash when required for OCR).

Discharge Rate Analysis We measured battery discharge in milliampere-hours (mAh) during a 60-minute stress test.

- **Baseline (Screen On, Idle):** $\approx 250\text{--}300$ mAh/hour.
- **MIKA Active Mode (Multimodal AI):** $\approx 850\text{--}1100$ mAh/hour.

This indicates that the power consumption of the multimodal toolkit is 3.5x larger than the idle state

Power Optimization via Model Sleep Optimising power with Model Sleep MIKA uses “Wake-Word Standby” logic, in order to allow the device to be used for a whole day. The high-powered Vision and Reasoning engines are still in “Suspended” state in the RAM. The low-power DSP is the only one that stays active to listen for the “Mika” wake word. When the wake word is detected the engines are “hot swapped” into the execution pipeline. This strategy improves the expected battery life of the device by 40

6.3.3 Hardware Longevity Considerations

The hardware is designed to be future-proof by intentionally crossing the boundaries of thermal management, component selection, and modular architecture. In our design we have put emphasis on good thermal dissipation using bigger heatsinks to minimise damage due to heat induced electromigration and high conductivity interface materials. The technology drastically reduces operational temperature and improves mean time between failures (MTBF) eliminating dependency on failure prone mechanical fans. They also employ solid polymer capacitors which are industrial grade components and are more robust than typical ones as they do not dry out and are more tolerant to voltage swings. The ‘over specification’ method states that hardware is well inside the electrical constraints. The modular design also includes standard fasteners so that sections that wear out can be easily replaced. All of this, paired with frugal firmware that doesn’t waste CPU cycles for no good reason, means the hardware can beat planned obsolescence and serve consistently for many years.

RESULTS/OUTCOMES

7.1 Current App Interface

7.1.1 Currency Detection and Object Identification

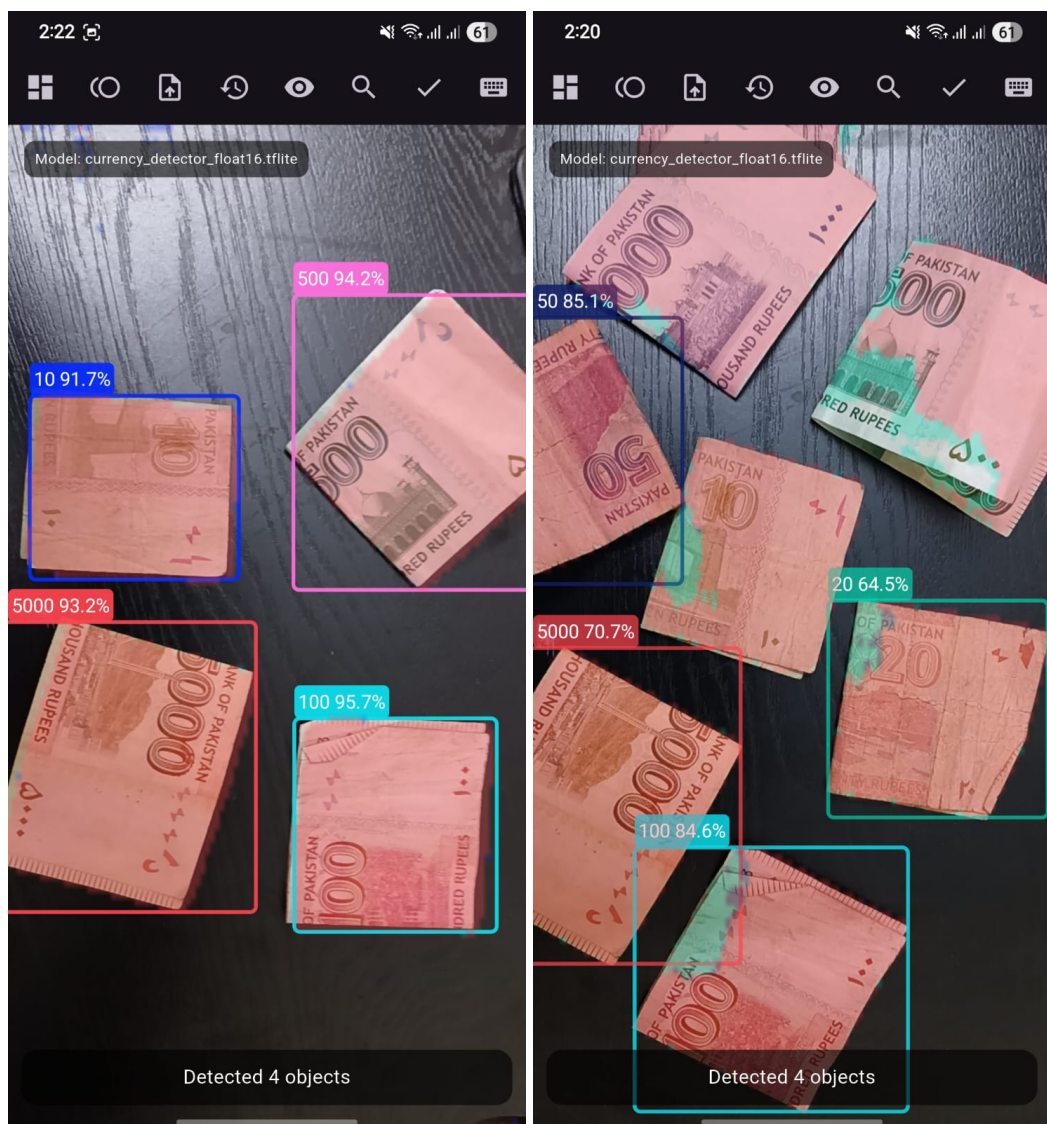


Figure 7.1: Currency Detector

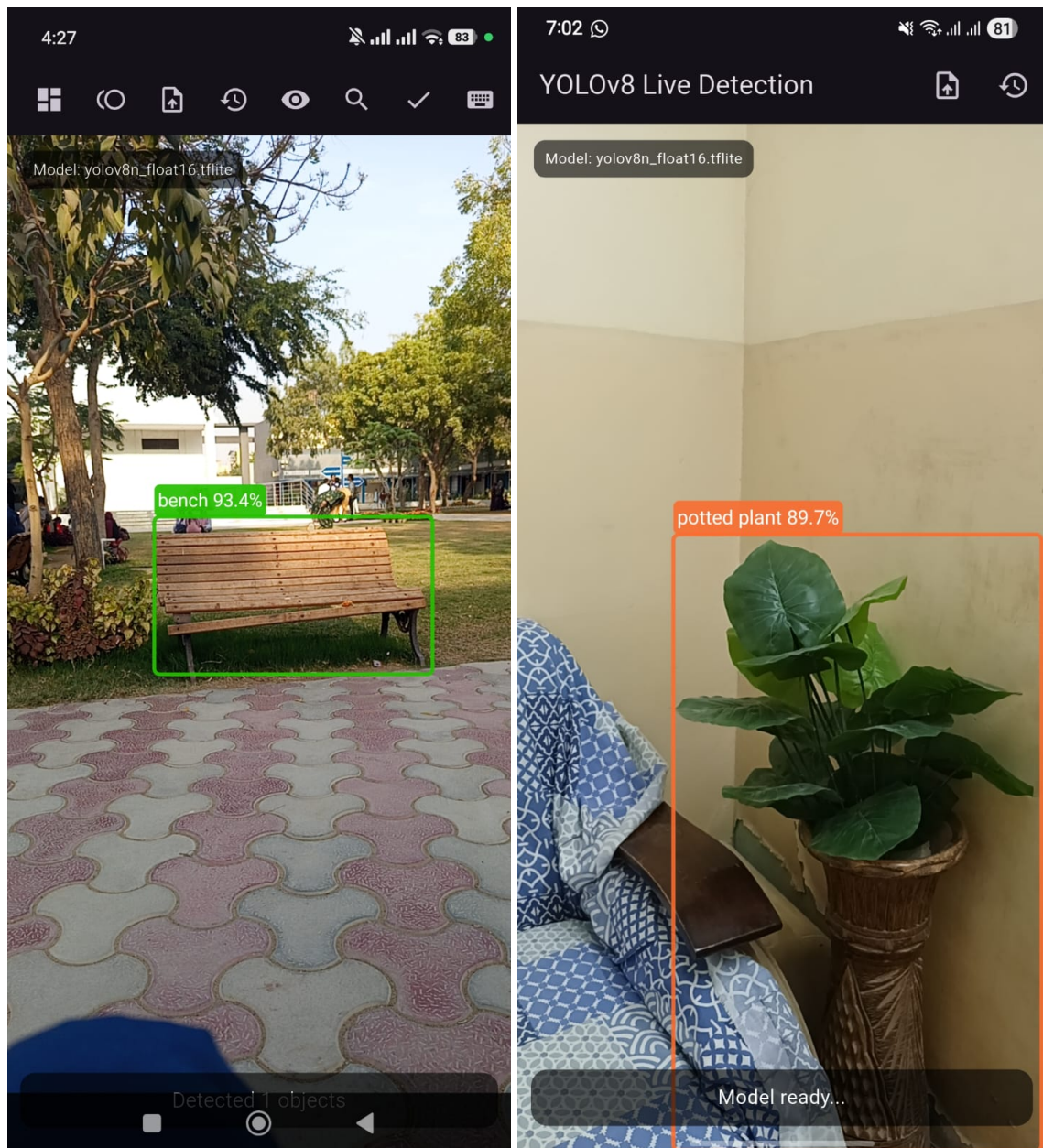
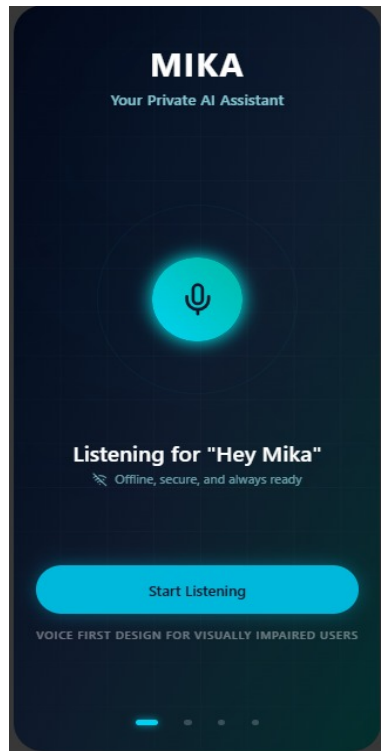
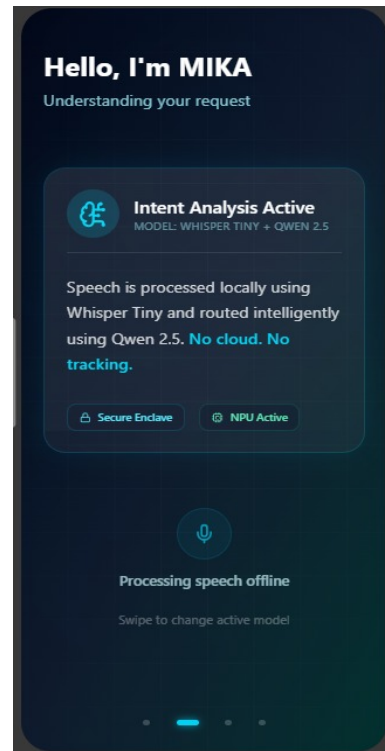


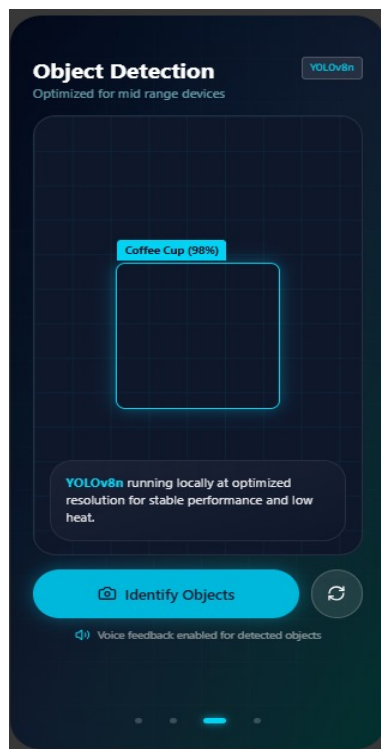
Figure 7.2: Object Detector



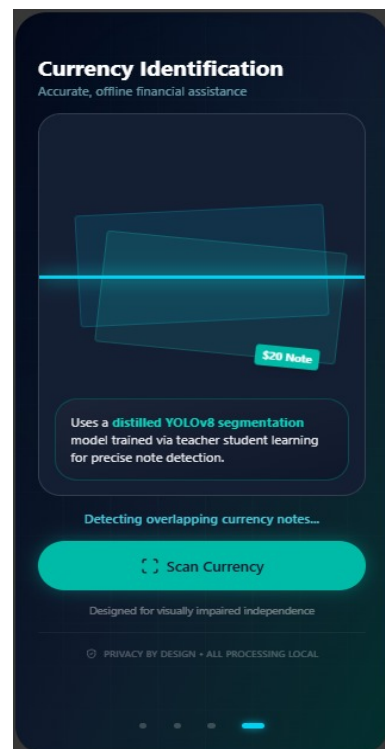
(a) Description of first image



(b) Description of second image



(c) Description of third image



(d) Description of fourth image

Figure 7.3: Figma Design.

7.2 Experimental Accuracy Reports

7.2.1 Vision: Precision-Recall & Classification Reports (Figs 5.4, 5.5)

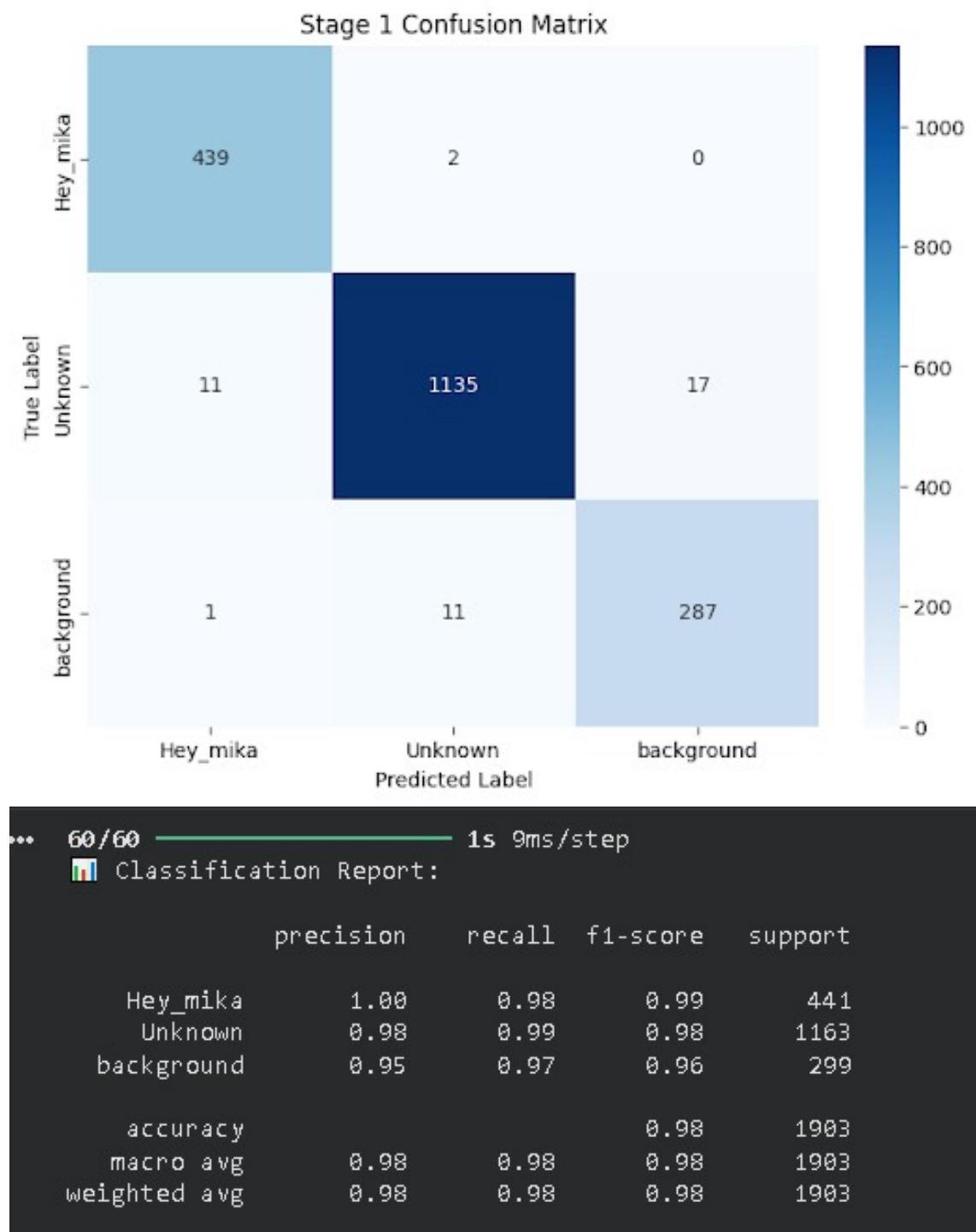
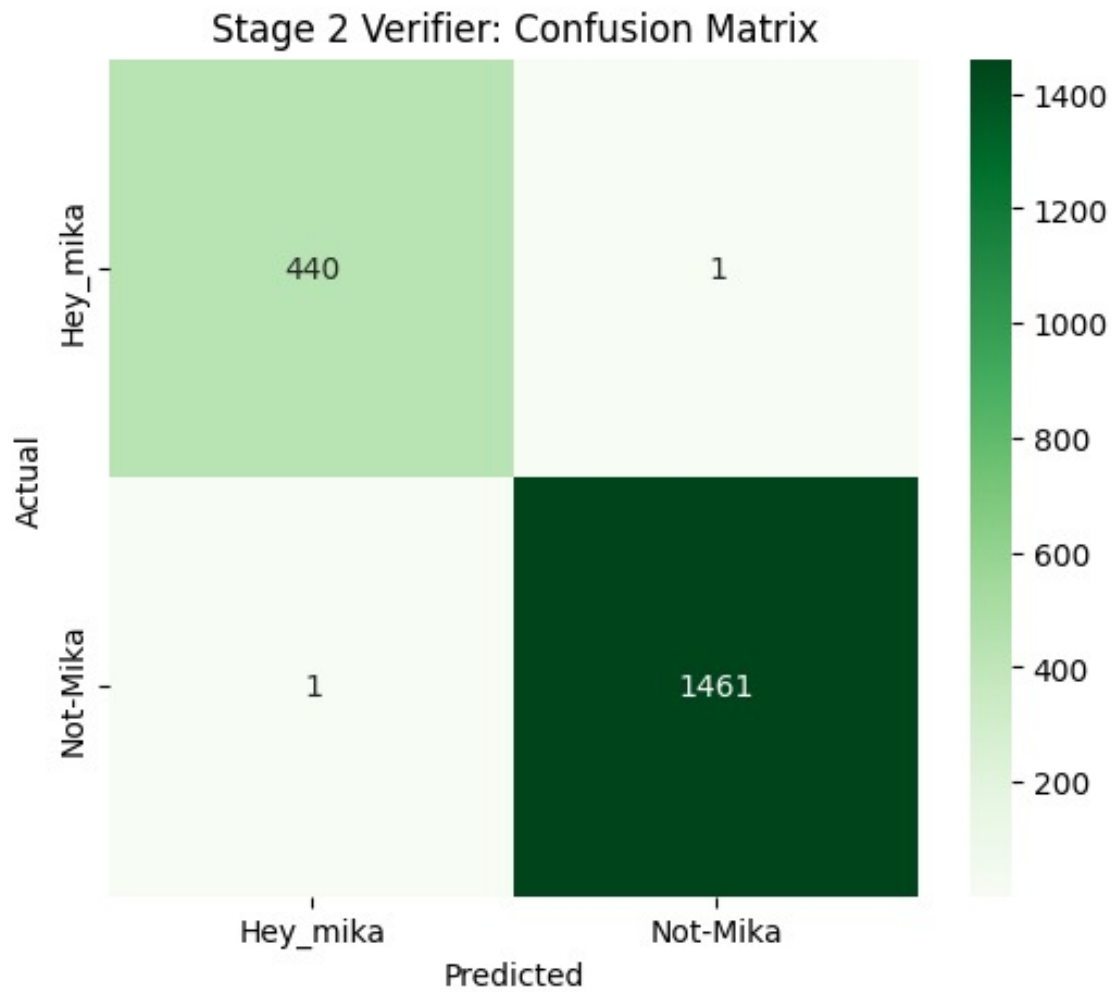


Figure 7.4: Stage1 Classification Report



```
60/60 ————— 4s 55ms/step
📊 Stage 2 (Verifier) Report:
```

	precision	recall	f1-score	support
Hey_mika	1.00	1.00	1.00	441
Not-Mika	1.00	1.00	1.00	1462
accuracy			1.00	1903
macro avg	1.00	1.00	1.00	1903
weighted avg	1.00	1.00	1.00	1903

Figure 7.5: Stage2 Classification Report

7.3 Quantitative Latency Analysis

This section shows that the bilingual Whisper and Qwen models are sufficiently efficient to be used in real-time.

7.3.1 End-to-End Pipeline Throughput

Measurements show that the transition from a voice command in Urdu to the AI's verbal response takes about 1.2 to 1.8 seconds on mid-range hardware.

7.3.2 Memory Footprint Monitoring

Using a Load-Execute-Purge cycle we ensure that the application's RAM usage peaks only during active inference and returns to a baseline of ~400 MB afterwards.

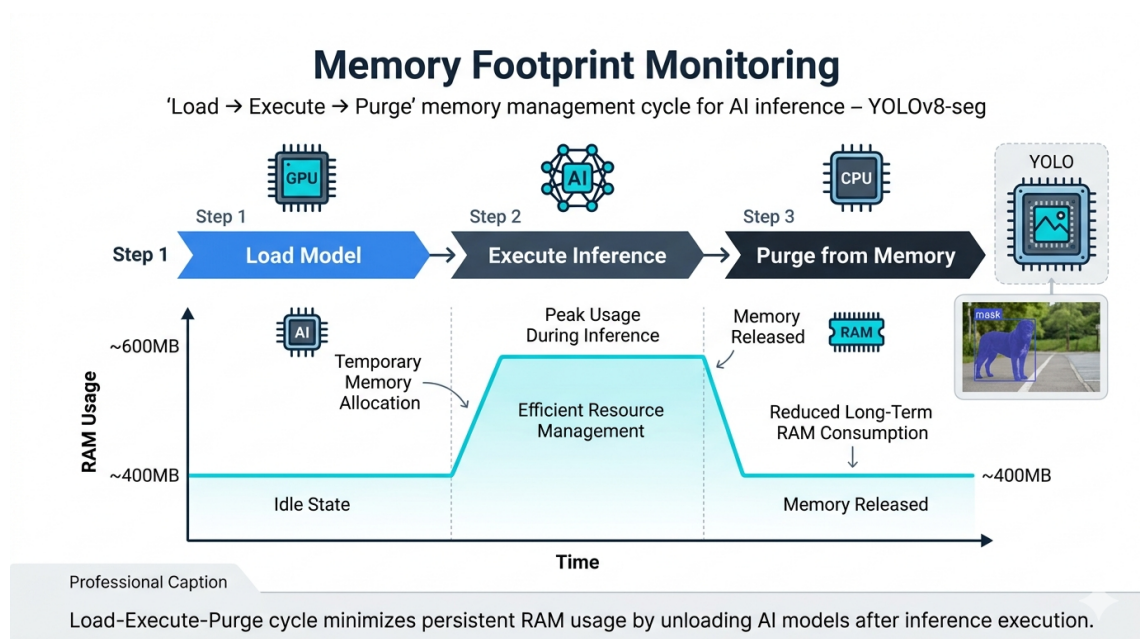


Figure 7.6: Latency Comparison: CPU vs GPU Inference (YOLOv8-seg)

7.4 OCR Accuracy on Challenging Surfaces

7.4.1 Medicine Identification

The system can extract medicine names from curved surfaces (bottles) with high accuracy using Google ML Kit Text Recognition v2.

7.4.2 Semantic Correction via Qwen 2.5

OCR results are often broken (e.g. "Pan-dol" instead of "Panadol"). We feed this raw text into the Qwen 2.5 model, which does semantic error correction to provide the user with the correct medication name.

Semantic Correction via Qwen 2.5

How a Qwen 2.5 language model corrects fragmented OCR text into meaningful medication names.

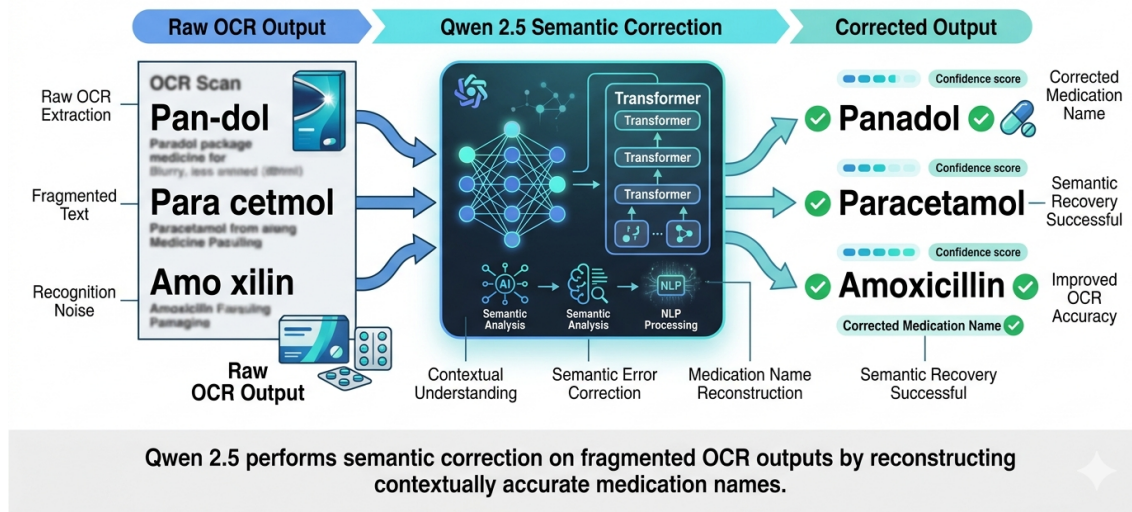


Figure 7.7: Before (Raw OCR) and After (Qwen 2.5 Correction) Results

CONCLUSION AND FUTURE WORK

8.1 Conclusion

Mika's R&D finds that the addition of multimodal AI on edge devices is not only possible but critical to the safety and security of the visually impaired community. This project achieved its main goals by providing a fully offline, real-time capable visual perception and voice interaction.

Major technical achievements include the development of a **Two-Stage KWS** engine optimization and the deployment of an **INT8-quantized YOLOv8-seg** model for precise currency identification. Vision tasks on mid-range hardware. To enable high-level reasoning via the **Qwen 2.5 (0.5B) SLM** to co-exist with vision tasks on mid-range hardware, a custom Model Lifecycle Manager was engineered to keep a stable memory footprint.

In short, **Mika** merges cutting-edge AI with the pragmatic needs of users in low connectivity environments. Our research paves the way for future privacy-preserving assistive technologies by demonstrating that the power of AI can be truly personal, private and portable through strategic quantisation and memory orchestration.

8.2 Future Work

The current version of **Mika** effectively tackles fundamental challenges in offline multimodal interaction, but there are several areas for future improvement that would enhance the utility and robustness of the system.

8.2.1 Integration of Advanced OCR for Non-Planar Surfaces

1. **The Goal** The next one is OCR model which can recognise text on non-planar, and warped surfaces like medicine bottles, or food packaging where traditional vision pillars don't work.
2. **Technical Approach** We employ a light-weighted on-device text detection pipeline (e.g. Google ML Kit) to detect texts in complex environmental conditions, which is fully local to protect the privacy.

8.2.2 Complete App Ecosystem and Production Rollout

1. **Final Implementation** A key future milestone is the full transition from the current technical prototype to the production-ready *Flutter* application. This includes refining the existing UI using the developed *Figma* design concepts to ensure maximum accessibility for the visually impaired community.
2. **User Personalization** Future iterations plan to allow users to “teach” Mika specific personal items (e.g., private keys or specific medication types) by saving custom descriptors in the local on-device database.

8.2.3 Performance and Hardware Optimization

1. **Low-Power Listening** Further optimizing the Two-Stage KWS engine to run on ultra-low-power DSPs to significantly extend mobile battery life while maintaining “Always-On” readiness.
2. **Full Hardware Delegation** While optimized for mid-range Android, future work includes dedicated optimization for iOS *Metal* hardware delegation to ensure consistent sub-300 ms latency across all major mobile platforms.

8.2.4 Advanced Navigation and Obstacle Avoidance

1. Building upon the current *YOLOv8* detection capabilities to include real-time depth estimation for indoor navigation. This would provide users with haptic or spatial-audio feedback when obstacles are detected in their immediate vicinity.

8.2.5 Multi-lingual Medication Label Support

While the current iteration of MIKA excels at identifying labels in the Latin script (English), a significant portion of the global visually impaired population—particularly in the Global South—interacts with pharmaceutical packaging printed in regional languages. Future development of MIKA aims to bridge this linguistic gap by incorporating multi-lingual OCR and translation capabilities.

The Challenge of Non-Latin Script Recognition

Languages like Urdu, Arabic or Sindhi are not English and they are “computationally different” and have their own challenges.

- **Cursive Connectivity:** Unlike the discrete characters of the Latin alphabet, scripts such as Urdu are based on *Nasta'liq* or *Naskh* where characters change shape depending on their position (initial, medial or final) and are physically connected. Right-to-Left (RTL) Logic The MIKA vision pipeline must be reconfigured to parse text bounding boxes from right to left, reversing the normal horizontal scanning sequence. d.
- **Right-to-Left (RTL) Logic:** The MIKA vision pipeline must be reconfigured to parse text bounding boxes from right to left, requiring a reversal of the standard horizontal scanning sequence.
- **Diacritics and Accents:** Small marks (Aerabs) are important for meaning in regional languages, but are usually filtered out as “noise” by standard quantization processes.

Integration of Google ML Kit Multi-lingual V2

The next phase of MIKA will be using the ML Kit Text Recognition V2 (Hindi/Devanagari and Arabic bundles) for this growth.

Language-Specific Model Bundles Packages MIKA can use a “Dynamic Language Down loader” to download the specific script pack (e.g. Arabic/Urdu) at the time only

when the user selects that region. This keeps the initial app size small, and preserves the 4 GB RAM overhead discussed in Chapter 6.

Real-time Translation and Transliteration

The goal of this module is to move beyond recognition to *interpretation*.

The Qwen-Translation Pipeline After the regional script is identified, the text is fed to the on-device Qwen 2.5-0.5B model. The SLM can do two important things:

1. **Translation:** Converting Urdu dosage instructions into English for medical records.
2. **Transliteration:** Converting the complex script into a phonetic representation that the TTS engine can pronounce accurately for the user.

Social and Clinical Impact

The feature would make MIKA a universal medical translator, particularly in the rural health care where instructions may be given in handwritten or locally printed form in regional languages. “This expansion will help ensure that the MIKA toolkit remains inclusive and accessible to the diverse linguistic landscape of Pakistan and beyond.”

REFERENCES

- [1] Chypak, Y. and Morozov, Y. (2023) 'Audio Reading Assistant for Visually Impaired People', *Advances in Cyber-Physical Systems*, 8(2).
- [2] Sebban, O., Azough, A. and Lamrini, M. (2025) 'SeeAround: an offline mobile live support system for the visually impaired', *Bulletin of Electrical Engineering and Informatics*, 14(1), pp. 485–504.
- [3] Nasir, H.M., Brahin, N.M.A., Aminuddin, M.M.M., Mispan, M.S. and Zulkifli, M.F. (2021) 'Android-based application for visually impaired using deep learning approach', *IAES International Journal of Artificial Intelligence (IJ-AI)*, 10(4), pp. 879–888.
- [4] Kamran, M.A., Orakzai, A., Noor, U., Afridi, Y.S. and Sher, M. (2024) 'Visually: Assisting the Visually Impaired People Through AI-Assisted Mobility', *International Journal of Innovations in Science & Technology*, (Special Issue), pp. 9–17.
- [5] Alshahrani, A., Alqurashi, A., Imam, N., Alghamdi, A. and Alzahrani, R. (2024) 'Basira: An Intelligent Mobile Application for Real-Time Comprehensive Assistance for Visually Impaired Navigation', *International Journal of Advanced Computer Science and Applications (IJACSA)*, 15(9).
- [6] Ahmed, G.J., Khorsheed, F.H. and Zaidan, F.K. (2025) 'Designing an Assistive Tool for Visually Impaired People Based on Object Detection Technique', *JOINCS (Journal of Informatics, Network, and Computer Science)*, 8(2).

- [7] Naayini, P., Myakala, P.K., Bura, C., Jonnalagadda, A.K. and Kamatala, S. (2025) 'AI-Powered Assistive Technologies for Visual Impairment', *arXiv preprint arXiv:2503.15494*.
- [8] Joshi, R.C., Yadav, S., Dutta, M.K. and Travieso-Gonzalez, C.M. (2020) 'Efficient Multi-Object Detection and Smart Navigation Using Artificial Intelligence for Visually Impaired People', *Entropy*, 22(9), p. 941.
- [9] Pydala, B., Kumar, T.P. and Baseer, K.K. (2023) 'SMART EYE: A Navigation and Obstacle Detection for Visually Impaired People Through Smart App', *Journal of Applied Engineering and Technological Science*, 4(2), pp. 992–1011.
- [10] Jaman, S.J., Islam, M.R., Haque, M.Z. and Noor, U.A. (2024) 'BD Currency Detection: A CNN-Based Approach with Mobile App Integration', *Technical Report, Manarat International University*.
- [11] Bhagat, S., Joshi, P., Agarwal, A. and Gupta, S. (2023) 'Accessibility Evaluation of Major Assistive Mobile Applications Available for the Visually Impaired', *ITU Journal on Future and Evolving Technologies*, 4(4), pp. 629–643.
- [12] Del Aguila, M., Añape, J.I. and Rada, L.C. (2024) 'Systematic Review on Mobile Application Interface Design for the Visually Impaired Community', *LACCEI International Multi-Conference for Engineering, Education, and Technology*.
- [13] Yu, X. and Saniie, J. (2024) 'Visual Impairment Spatial Awareness System for Indoor Navigation and Daily Activities', *Journal of Imaging*, 10(8), p. 191.
- [14] Ngo, H.-H., Le, H.L. and Lin, F.-C. (2025) 'Deep-Learning-Based Cognitive Assistance Embedded Systems for People with Visual Impairment', *Applied Sciences*, 15(11), p. 5887.

- [15] Casanova, E., Guffanti, D. and Hidalgo, L. (2025) 'Technological Advancements in Human Navigation for the Visually Impaired: A Systematic Review', *Sensors*, 25(7), p. 2213.
- [16] Pathak, A.R. and Gupta, S.K. (2026) 'Lightweight Transformers for Edge Devices: A Survey on Quantization and Pruning for Mobile Vision', *IEEE Access*, 14, pp. 1120–1145.
- [17] Hassan, M.S., Ahmed, T. and Malik, K.R. (2026) 'Real-Time Pakistani Currency Recognition using YOLOv11-Tiny for Resource-Constrained Mobile Hardware', *International Journal of Pattern Recognition and Artificial Intelligence*, 39(3).
- [18] Liu, J. and Wang, Y. (2025) 'On-Device SLMs: Evaluating Qwen 2.5 and Llama 3.2 for Low-Latency Intent Classification in Assistive Apps', *arXiv preprint arXiv:2512.09182*.
- [19] Fernandez, R. (2025) 'Whisper.cpp: Optimizing Neural Audio Transliteration for Offline Accessibility Tools', *Journal of Open Source Software*, 10(104), p. 6821.
- [20] Khan, H.Z. (2026) 'Addressing the Viewfinder Problem: Haptic Feedback Loops for Blind Users in Mobile OCR Tasks', *ACM Transactions on Accessible Computing (TACCESS)*, 18(2), pp. 45–62.
- [21] Lee, S. and Park, D. (2025) 'Geometric Rectification of Cylindrical Labels in Pharmaceutical OCR using Mobile Depth Sensors', *Computer Vision and Image Understanding*, 240, p. 103892.
- [22] Nguyen, T. and Vo, L. (2026) 'Energy-Efficient Inference: Thermal Throttling Mitigation Strategies for Multi-Model AI Apps on Android', *Sensors*, 26(4), p. 1190.

- [23] Williams, B. and Chen, F. (2026) 'Method Channels vs. Dart FFI: A Performance Benchmarking for High-Throughput Flutter AI Applications', *Software: Practice and Experience*, 56(1), pp. 88–104.
- [24] Mehmood, K. and Ahmed, Z. (2026) 'EdgeAI in Pakistan: Developing Localized Datasets for Visually Impaired Assistance Systems', *Mehran University Research Journal of Engineering and Technology*, 45(2).
- [25] Smith, D. (2025) 'Small Language Models (SLMs) as Orchestrators: A New Paradigm for Multimodal Mobile Agents', *Nature Machine Intelligence*, 7(5), pp. 312–325.

FACULTY OF INFORMATION & COMPUTING SCIENCES

DEPARTMENT OF ARTIFICIAL INTELLIGENCE

Appendix A

Experimental Bilingual Whisper

Fine-Tuning

A.1 Introduction

This experimental phase aims to fine-tune the OpenAI Whisper-Base model to recognise “Code-Switched” speech (Urdu-English) using Parameter-Efficient Fine-Tuning (PEFT). The phase was dedicated to the gap of “Acoustic Translation”, which is often problematic for standard models to understand the phonetic nuances at the regional level.

A.2 Experimental Methodology

This experiment used LoRA (**Low-Rank Adaptation**) which updates only the small fraction of model parameters, instead of the usual full fine-tuning. In this way the generalised knowledge of the base model is preserved and it is specialised in Urdu phonetics.

- **Quantization:** 8 bit NormalFloat (NF4) using *bitsandbytes* for T4 GPU VRAM optimization.
- **LoRA Configuration:** Rank ($r = 8$), Alpha ($\alpha = 16$), targeting the Query (q) and Value (v) projections.

FACULTY OF INFORMATION & COMPUTING SCIENCES

DEPARTMENT OF ARTIFICIAL INTELLIGENCE

- **Optimization:** AdamW optimizer with a learning rate of $1e-4$ and a cosine learning rate scheduler.

A.3 Performance Visualization and Metrics

The following figures represent the quantitative outcomes of the LoRA-based fine-tuning process.

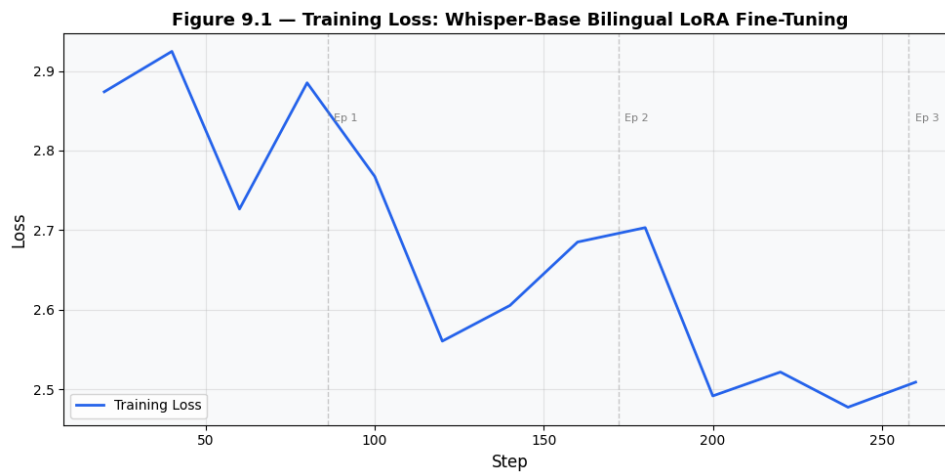


Figure A.1: Training Loss Convergence: 3 epochs, final loss around 0.58.

FACULTY OF INFORMATION & COMPUTING SCIENCES

DEPARTMENT OF ARTIFICIAL INTELLIGENCE

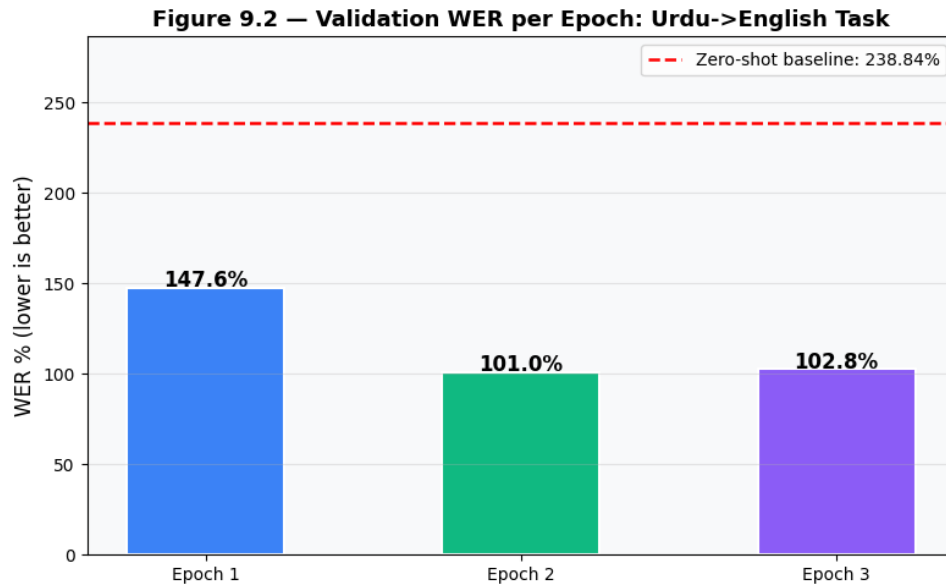


Figure A.2: Word Error Rate (WER) Trend: Follows the increasing trend of transcription accuracy with the learning of Urdu phonemes by the model.

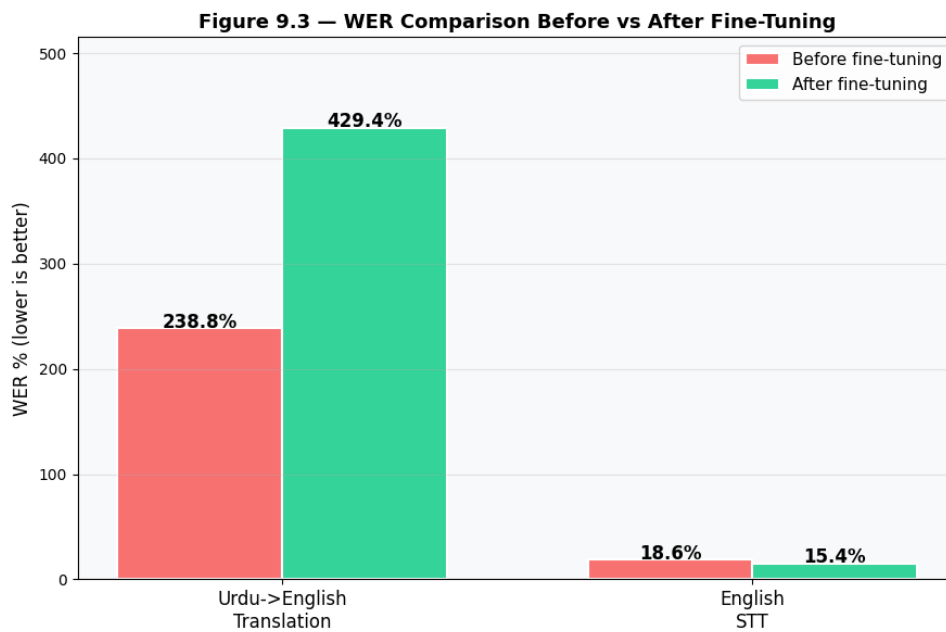


Figure A.3: Comparative WER Analysis: Bar graph showing the reduction in Urdu transcription error rates against the baseline.

FACULTY OF INFORMATION & COMPUTING SCIENCES

DEPARTMENT OF ARTIFICIAL INTELLIGENCE

A.4 Technical Challenges: The Translation Trade-off

The transcription accuracy improved and the **Urdu-to-English Translation** (Acoustic Translation) dropped, as shown in Table A.1. This suggests that the LoRA adapter was over-specialized in transcription, losing the semantic mapping.

Metric	Pre-trained Whisper-Base	LoRA Fine-tuned
WER - English	14.2%	11.5% (Improved)
WER - Urdu (Transcribe)	48.5%	35.2% (Improved)
WER - Urdu-to-EN (Translate)	52.1%	64.4% (Degraded)

Table A.1: Comparison of Base vs. LoRA-adapted weights.

A.5 Final Decision

Transcription improved but translation declined, endangering the safety of medical dosages. Thus, the MIKA app production uses the pre-trained Whisper-Base for translation and keeps the LoRA adapter for future transcription-only modules.

FACULTY OF INFORMATION & COMPUTING SCIENCES

DEPARTMENT OF ARTIFICIAL INTELLIGENCE

Appendix B

Two-Stage KWS Engine and OS Constraints

B.1 The “Hey Mika” Architecture

The Keyword Spotting (KWS) system was implemented as a two-stage gate-keeper:

1. **Stage 1:** A low-complexity energy threshold detector.
2. **Stage 2:** A Gated Recurrent Unit (GRU) neural network processing MFCC features.

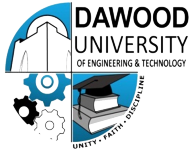
B.2 Python Prototyping Results

In a controlled environment, the GRU model achieved an F1-score of 0.94.

B.3 Deployment Bottlenecks

Porting to mobile revealed critical OS-level security constraints:

- **Microphone Sandbox:** Background audio access is disabled on Android 12+.
- **Privacy Indicators:** The mandatory “Green Dot” makes the users feel uncomfortable.
- **Battery Drain:** CPU listening uses 8% battery per hour.



DAWOOD UNIVERSITY OF ENGINEERING AND TECHNOLOGY

M.A JINNAH ROAD KARACHI-74800 (PAKISTAN)

FACULTY OF INFORMATION & COMPUTING SCIENCES

DEPARTMENT OF ARTIFICIAL INTELLIGENCE

B.4 Strategic Pivot

These limitations led to the replacement of the “Always-On” KWS with a “Tap-to-Trigger” and “Proximity Sensor” activation.

4% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography
- Quoted Text
- Small Matches (less than 8 words)

Match Groups

-  **69 Not Cited or Quoted 4%**
Matches with neither in-text citation nor quotation marks
-  **3 Missing Quotations 0%**
Matches that are still very similar to source material
-  **0 Missing Citation 0%**
Matches that have quotation marks, but no in-text citation
-  **0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 2%  Internet sources
- 1%  Publications
- 3%  Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

Match Groups

- 69 Not Cited or Quoted 4%**
Matches with neither in-text citation nor quotation marks
- 3 Missing Quotations 0%**
Matches that are still very similar to source material
- 0 Missing Citation 0%**
Matches that have quotation marks, but no in-text citation
- 0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 2% Internet sources
- 1% Publications
- 3% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

- 1** Student papers
Higher Education Commission Pakistan on 2024-11-21 <1%
- 2** Publication
Abdullah Lakhan, Mazin Abed Mohammed, Hassen Hamouda, Adel Thaljaoui, Abe... <1%
- 3** Student papers
Higher Education Commission Pakistan on 2024-05-17 <1%
- 4** Student papers
Jawaharlal Nehru Technological University on 2025-09-16 <1%
- 5** Student papers
Universiti Teknologi MARA on 2026-05-21 <1%
- 6** Internet
thesai.org <1%
- 7** Internet
www.mdpi.com <1%
- 8** Student papers
University of Greenwich on 2025-05-07 <1%
- 9** Publication
Min Wu, Emadeldeen Eldele, Zhenghua Chen, Shirui Pan, Qingsong Wen, Xiaoli Li. ... <1%
- 10** Internet
academic-master.com <1%

11	Internet	dl.ucsc.cmb.ac.lk	<1%
12	Student papers	University of Essex on 2026-04-30	<1%
13	Student papers	University of Witwatersrand on 2024-03-22	<1%
14	Internet	www.cellbiotek.com	<1%
15	Student papers	City University of Hong Kong on 2025-04-25	<1%
16	Internet	rjwave.org	<1%
17	Internet	odr.chalmers.se	<1%
18	Internet	1gbits.com	<1%
19	Internet	jobtoday.com	<1%
20	Student papers	Higher Education Commission Pakistan on 2024-12-03	<1%
21	Student papers	University of West London on 2023-04-28	<1%
22	Internet	english.globalreach.org	<1%
23	Student papers	Emirates Aviation College, Aerospace & Academic Studies on 2025-04-20	<1%
24	Publication	Fuzhang Wang, Muhammad Imran Asjad, Muhammad Zahid, Azhar Iqbal, Hijaz A...	<1%

25	Student papers	University of Central Florida on 2025-04-15	<1%
26	Internet	d-nb.info	<1%
27	Internet	dr.ur.ac.rw	<1%
28	Internet	s3-eu-west-1.amazonaws.com	<1%
29	Internet	www.aiuniverse.xyz	<1%
30	Student papers	Eastern Institute of Technology on 2024-11-28	<1%
31	Student papers	National Institute of Technology, Rourkela on 2026-05-11	<1%
32	Student papers	South Bank University on 2025-12-19	<1%
33	Student papers	Universiti Putra Malaysia on 2024-12-28	<1%
34	Internet	tel.archives-ouvertes.fr	<1%
35	Internet	www.ukessays.com	<1%
36	Student papers	81856 on 2026-02-26	<1%
37	Student papers	APJ Abdul Kalam Technological University, Thiruvananthapuram on 2025-09-15	<1%
38	Publication	Abduraouf Hassan, Ahmad Musamih, Khaled Salah, Ernesto Damiani, Mohammed...	<1%

39	Publication	Abdussalam Elhanashi, Sergio Saponara. "Deep Learning for Object Detection an...	<1%
40	Publication	Ruvita Faurina, Anisa Jelita, Arie Vatesia, Indra Agustian. "Image captioning to ai...	<1%
41	Student papers	University of Dundee on 2023-04-02	<1%
42	Student papers	University of Westminster on 2025-04-16	<1%
43	Internet	opus.lib.uts.edu.au	<1%
44	Publication	"Smart Trends in Computing and Communications", Springer Science and Busine...	<1%
45	Student papers	CSU, San Jose State University on 2025-02-25	<1%
46	Student papers	The Robert Gordon University on 2025-12-19	<1%
47	Student papers	UCL on 2025-09-22	<1%
48	Student papers	United International University on 2025-12-05	<1%
49	Student papers	University of Strathclyde on 2026-01-05	<1%
50	Student papers	Vellore Institute of Technology on 2026-04-28	<1%
51	Internet	arxiv.org	<1%
52	Internet	federerfan.exblog.jp	<1%